

Performance Engineering (2360013) — Assignment 1

Student: חביב ראמי · ID 325420180 **Instructor:** Nadav Amit · Spring 2026 **Environment:** provided VM course-08 — KVM guest, Intel Xeon Gold 5420+, 4 vCPUs, kernel 7.0.0-15-generic, ext4, 7.7 GiB RAM. Kernel sources at `/usr/src/linux-7.0.0`.

How this investigation was run

Each phenomenon was driven as a hypothesis loop: establish a baseline **with run-to-run variance**, predict before measuring, find the single measurement that separates competing hypotheses, **rule out alternatives with data**, and **confirm the mechanism against the kernel source under `/usr/src`**. Raw tool output for every probe is filed under `task1/` and `task2/` (referenced inline by filename); the full timestamped trail is in the git history.

Results at a glance

- **Task 1 — pipe latency improves under CPU load.** Cause: **scheduler wake-placement**. With idle CPUs available, `select_idle_sibling()` spreads the two pipe partners onto *different* cores, making every round-trip a cross-core wakeup (~11 μ s); background load removes the idle CPUs, so the pair is co-located on one core (~5 μ s). Frequency (DVFS) and idle/HLT cost were ruled out with data; confirmed at `kernel/sched/fair.c:select_idle_sibling`.
 - **Task 2 — v2 (4 MiB prepare) reads ~9 % faster (reproduced).** Cause: **prepare block size sets page-cache folio order** — v2's 4 MiB writes build 2 MiB PMD-order folios, v1 stays at 16 KiB. v2's folios are **mapped as 2 MiB huge pages** (`FilePmdMapped = 64 MiB`, `v1 = 0`), which **collapses dTLB misses ~475 $\times$ (8.79 M $\rightarrow$ 18 K)**; since the random read is dTLB-bound, that is the +9–10 %. It is a TLB effect (LLC misses equal), not disk (zero I/O). Confirmed in `mm/memory.c:do_set_pmd` and `mm/readahead.c:page_cache_ra_order`. *(The effect was initially hidden by memory fragmentation after 22 days of VM uptime — a reboot restored it; see Task 2 §7.)*
-
-

Task 1 — Why pipe latency *improves* under background CPU load

Phenomenon. `lat_pipe` (lmbench "hot-potato" Unix-pipe round-trip) reports ~11 μ s idle, but ~5 μ s when `stress-ng --cpu 3` runs alongside it — a ~2 $\times$ *speed-up* from adding load.

Result (one line). The cause is **scheduler wake-placement**, not frequency or idle/halt cost. When idle CPUs exist, the scheduler spreads the two pipe partners onto *different* cores, so every round-trip pays a cross-core wakeup (reschedule IPI + cache-line bounce). Background load removes the idle CPUs, so the scheduler co-locates the pair on one core, turning each round-trip into a cheap same-core context switch. Confirmed in `kernel/sched/fair.c`, `select_idle_sibling()` line 110 (`select_idle_cpu`).

1. Environment (as shipped — recorded before changing anything)

`task1/env_asshipped.txt`

- **KVM full-virt guest** (`systemd-detect-virt = kvm`), Intel Xeon Gold 5420+, **4 vCPUs**, 1 thread/core, each vCPU presented as its own socket; 1 NUMA node. Kernel 7.0.0-15-generic.
- **No cpufreq driver** in the guest (`cpupower frequency-info: "no or unknown cpufreq driver active"; governors Not Available`). `cpu MHz` fixed at 2000.
- **No cpuidle states** (`cpupower idle-info: driver none, "No idle states"`).
- `perf_event_paranoid = -1` (full perf access incl. kernel).

Methodological note (lecturer slide 65 vs our setup). The deck recommends pinning the governor to `performance` and disabling idle states for a stable benchmark. Those knobs **do not exist inside this guest**, so we instead (a) recorded the environment as shipped, (b) reproduced the effect, then (c) controlled one variable at a time, and verified frequency was constant *via counters* rather than by pinning. This is a reasoned deviation, not an oversight.

2. Baseline + variance

`logs/task1_20260610_1427.log` (`lat_pipe` prints to stderr; numbers are from the session log)

condition	n	median	min-max
no load	10	11.12 μs	9.88 – 11.20
<code>stress-ng --cpu 3</code>	10	5.26 μs	4.74 – 6.39

Effect = **2.1 $\times$ (5.9 μ s)**, far larger than the run-to-run spread. The effect is real and stable.

3. Three hypotheses, and how each was settled

We compared the good (loaded, fast) and bad (idle, slow) cases and drilled down from counters to controlled experiments to source.

F — CPU frequency scaling (DVFS). RULED OUT.

task1/p2_perfstat_{noload,load}.txt

perf stat frequency, from `cycles ÷ ref-cycles` (a ratio, so immune to the fact that `lat_pipe` self-calibrated to different iteration counts under contention):

	cycles	ref-cycles	ratio = freq/nominal
no load	1.620 B	1.203 B	1.347
with load	73.485 B	54.567 B	1.347

Identical → when the core actually executes, it runs at the same ~2.7 GHz in both conditions. Frequency does not change. **F is out.** (The raw `perf stat` latency of ~69 μs is measurement overhead and is *not* compared across runs — only the ratio is.)

I — Idle / HLT→VMEXIT wakeup cost. RULED OUT.

task1/p2b_pin2core_idle.txt, task1/p2b_pin2core_busy.txt

Initial (wrong) reading: pinning *both* pipe ends to one core reproduced the speed-up (`p2_pin1core_noload.txt`: 5.42 μs, no load) — suggesting idle removal was the cause. But that probe changed **two** variables at once (no-idle *and* same-core). A placement-controlled experiment (Phase 2B) fixed placement = cross-core and flipped only idle:

placement	cores idle between turns?	latency
cross-core (cpus 0,1)	yes	11.2 μs
cross-core (cpus 0,1) + a spinner pinned to each	no	10.9 μs

Keeping the cores busy with placement held cross-core did **not** help. **I is out.** (Confound handled: the no-spinner case (no contention) and the spinner case (contention) are *both* ~11 μs, so the slowness is the cross-core placement, not CPU contention.)

P — Scheduler wake-placement. RULED IN.

task1/p2_pin1core_noload.txt + Phase 2B + Phase 4 evidence

Holding the *other* variable (idle) fixed and flipping placement:

placement	core state	latency
same core (pin 1)	busy	5.4 μs
cross core (pin 2, idle or busy)	either	~11 μs
loaded, unpinned (scheduler chose)	busy	5.3 μs
idle, unpinned (scheduler chose)	idle	11.1 μs

Same-core ⇒ fast, cross-core ⇒ slow, independent of idle/busy. **Placement is the axis.**

4. Direct evidence (histogram + sampled callgraph)

Off-CPU latency histogram (bpftrace on `sched_switch`) — `task1/p4_offcpu_hist_*.txt`. Time each pipe end stays blocked per handoff:

```
no load:   mode [8, 16) µs   (139,094 hits)      with load: mode [2, 4) µs   (311,579 hits)
           [16,32) µs 33,289                        [4, 8) µs  40,707
```

The whole distribution drops $\sim 4\times$, matching the $11 \rightarrow 5 \mu\text{s}$ latency.

Sampled callgraph (`perf record -e sched:sched_switch -g`) — `task1/p4_callgraph_noload.txt`:

```
16.63% lat_pipe S ==> swapper/0:0    ← end #1 blocks; cpu0 goes idle
14.74% lat_pipe S ==> swapper/3:0    ← end #2 blocks; cpu3 goes idle
anon_pipe_read → schedule → __schedule
```

No-load, the two ends sit on **separate cores (0 and 3)** and each blocks to the idle task (`swapper`) — i.e. the pair is spread, and every resume is a cross-core wakeup. This is the mechanism made visible.

(The per-CPU switch-count files `p2b_latpipe_cpudist_*` are aggregated over many iterations and are inconclusive about momentary co-location; treated as weak supporting data only.)

5. Kernel-source confirmation — `kernel/sched/fair.c`

`task1/p4_select_task_rq_fair.txt` (`fair.c:8579`), `task1/p4_select_idle_sibling.txt` (`fair.c:7836`)

Wakeup placement path: `try_to_wake_up` → `select_task_rq` → `select_task_rq_fair`. For a normal task wakeup it takes the **fast path**, `select_task_rq_fair` line 64:

```
new_cpu = select_idle_sibling(p, prev_cpu, new_cpu);
```

Inside `select_idle_sibling`, the early returns use `target / prev` *only if already idle* (lines 24–26, 31–37). The decisive step is **line 110**:

```
i = select_idle_cpu(p, sd, has_idle_core, target); /* scan LLC domain for ANY idle CPU */
if ((unsigned)i < nr_cpumask_bits)
    return i;
```

- **Idle system (no bg load):** `select_idle_cpu` finds an idle CPU and returns it → the woken pipe partner is placed on a *different* core than the waker → the pair is split across cores → each hot-potato round-trip is a cross-core wakeup (reschedule IPI + pipe-buffer/`task_struct` cache-line bounce between cores) $\approx 11 \mu\text{s}$.
- **Loaded system:** `select_idle_cpu` finds **no** idle CPU (falls past line 111) → the function returns `prev / target` → the partner is **co-located with the waker** → same-core context switch, no IPI, no cross-core cache traffic $\approx 5 \mu\text{s}$.

So background load helps **by accident**: it removes the idle CPUs that `select_idle_cpu` would otherwise hand the wakee, forcing co-location.

6. Conclusion

The pipe benchmark is faster under CPU load because the Linux scheduler's idle-CPU-seeking wakeup placement (`select_idle_sibling / select_idle_cpu`) **spreads** the two communicating processes across cores when CPUs are idle, making every round-trip a cross-core wakeup; load removes the idle CPUs and forces same-core co-location, which is $\sim 2\times$ cheaper. Frequency scaling (F) was ruled out by a

constant `cycles/ref-cycles` ratio; idle/HLT cost (I) was ruled out because cross-core-but-busy stayed slow. The mechanism was confirmed by a placement-controlled experiment that toggles the effect, an off-CPU latency histogram, a sampled callgraph, and the kernel source.

Evidence index

File	Evidence
<code>task1/env_asshipped.txt</code>	environment (KVM, no cpufreq/cpuidle)
<code>logs/task1_20260610_1427.log</code>	baseline ×10 each condition
<code>task1/p2_perfstat_{noload,load}.txt</code>	F ruled out (freq ratio constant)
<code>task1/p2_pin1core_noload.txt</code>	same-core = fast
<code>task1/p2b_pin2core_{idle,busy}.txt</code>	I ruled out (cross-core busy still slow)
<code>task1/p4_offcpu_hist_{noload,load}.txt</code>	off-CPU latency histogram
<code>task1/p4_callgraph_noload.txt</code>	sampled callgraph (pair on separate cores → swapper)
<code>task1/p4_select_idle_sibling.txt</code> , <code>p4_select_task_rq_fair.txt</code>	kernel source

Task 2 — Root cause of the v1/v2 random-read gap (prepare block size)

Setup. Same 64 MiB file built two ways — v1 with default (~16 KiB) prepare writes, v2 with 4 MiB writes — then the identical random-read (`mmap`, 4 KiB, 5 s) benchmark on each.

Result (one line). The prepare block size sets the **page-cache folio order**: v2's 4 MiB writes build the whole file out of **2 MiB (PMD-order) folios**, v1's small writes cap at 16 KiB. v2's huge folios get **mapped as 2 MiB huge pages** (`FilePmdMapped` = 64 MiB, the entire file; v1 = 0), which **collapses dTLB misses ~475x (8.79 M → 18 K)** — and since this benchmark is dTLB-bound, that is the **+9–10 %** speedup. Ruled out: disk fragmentation (zero disk I/O). Confirmed against `mm/memory.c:do_set_pmd` and `mm/readahead.c:page_cache_ra_order`.

Investigative note. The gap initially did **not** reproduce on the lab VM. We traced that to **memory fragmentation after 22 days of uptime** preventing the huge-page allocation (`do_set_pmd` never fired, `FilePmdMapped` = 0, anonymous THP also refused to engage). A reboot defragmented memory and the gap appeared immediately — see §7. The mechanism below is from the reproduced run.

1. Environment [task2/p0_env.txt]

KVM guest, kernel 7.0.0-15, `$HOME` on **ext4**, THP = `[always]`, sysbench 1.0.20.

2. The gap — reproduced [task2/p10_reboot_confirmed.txt]

	v1 reads/s	v2 reads/s	v2 vs v1
fresh-boot run	1,942,670	2,124,442	+9.4 %

Matches the assignment's ~8–9 %. (On the long-uptime VM it measured flat — §7.)

3. Rule out the obvious wrong answer — disk fragmentation [p1b_filefrag, p2_perfstat_*]

- `filefrag`: v1 = 2 extents, v2 = 1 — trivial.
- `perf stat`: **page-faults == minor-faults exactly** (zero major faults) → the file is 100 % page-cached, **no disk I/O during the run**. A run that never touches disk cannot care about disk layout. **Fragmentation ruled out, with data.**

4. The mechanism — confirmed end to end

Step 1: prepare block size → folio order [p5_folio_hist_]. *Folio-order histogram (bpftime on `mm_filemap_add_to_page_cache`, captured during the buffered-write* prepare):*

	folios built	
v1 (4 KiB writes)	4096 × order-2 (16 KiB)	small folios only
v2 (4 MiB writes)	32 × order-9 (2 MiB / PMD) = whole file	PMD-order folios

Step 2: huge folios get PMD (huge-page) mapped [p10_reboot_confirmed.txt]. `smaps_rollup` during a run:

	FilePmdMapped
v1	0 kB (16 KiB folios are 4 KiB-PTE mapped)
v2	65,536 kB = the entire 64 MiB file (2 MiB folios mapped as PMD huge pages)

Step 3: huge mapping collapses dTLB misses → the speedup [p10_reboot_confirmed.txt]. `perf stat`, same random-read pattern:

	reads/s	dTLB-load-misses	cache-misses (LLC)
v1	1,886,428	8,790,240 (≈ 0.93 / read)	130.6 M
v2	2,082,856	18,488 (≈ 0 / read; ~475× fewer)	126.8 M ($\approx$ equal)

The benchmark is **dTLB-bound** — a 64 MiB working set with 4 KiB pages thrashes the TLB (4 KiB reach ≈ 4 MiB $\ll$ 64 MiB), so v1 misses on nearly every random read. v2's 2 MiB pages cover the file in ~ 32 TLB entries → essentially **zero** dTLB misses → no page-walk stalls → more reads in the same cycles → **+9–10 %**. The `cache-misses` (LLC) counter is **equal** between v1 and v2, which proves the win is the **TLB**, not the cache.

4b. Independent magnitude check [p5c_force_thp.txt]

Running the same random-read pattern over an explicit 2 MiB mapping (`MAP_HUGETLB`) vs 4 KiB pages gave dTLB-misses ~ 194 M → ~ 3 K and **+12.7 % (256 MiB) / +20.2 % (1 GiB)** — confirming, independently of the file path, that 2 MiB mapping of a TLB-bound random read is worth this order of magnitude.

5. Kernel-source confirmation [p4_page_cache_ra_order, p4_do_set_pmd]

- **Folio order follows the I/O size** — `mm/readahead.c:467 page_cache_ra_order()`: `new_order = min(mapping_max_folio_order(mapping), ilog2(ra->size))` — the folio order is bounded by the request size. The buffered-write path that built our folios applies the same rule, so v2's 4 MiB writes reach order-9 (2 MiB) and v1's small writes stay $\leq$ order-2. (§4 Step 1.)
- **The huge mapping** — `mm/memory.c:5408 do_set_pmd()` installs a 2 MiB PMD when the folio is PMD-order and the mapping is PMD-aligned. For v2 it fires (→ `FilePmdMapped = 64 MiB`); for v1 the 16 KiB folio fails the `folio_order == HPAGE_PMD_ORDER` check and is PTE-mapped. (§4 Step 2.)

6. Conclusion

Root cause: the prepare write block size sets the page-cache folio order — v2's 4 MiB writes build 2 MiB PMD-order folios, v1's stay at 16 KiB. v2's folios are then **mapped as 2 MiB huge pages** (`FilePmdMapped = 64 MiB`), which **eliminates the dTLB misses** that dominate this random-read benchmark (8.79 M → 18 K), yielding the **+9–10 %** gap. It is a **TLB** effect, not a cache effect (LLC misses

are equal) and not a disk effect (zero disk I/O). Every link is measured: folio order (histogram) → PMD mapping (`FilePmdMapped`) → dTLB collapse (`perf stat`) → throughput → confirmed in `do_set_pmd / page_cache_ra_order`.

7. Why it first didn't reproduce — memory fragmentation (the trail)

On the lab VM (22 days uptime) the gap was **absent**: $v1 \approx v2$, every counter flat, `FilePmdMapped = 0`, `do_set_pmd` never fired, and anonymous THP also refused to engage (`AnonHugePages = 0` despite `THP=always`, `MADV_COLLAPSE` → `EINVAL`). Those are the classic signatures of **fragmented memory** — the kernel cannot find 2 MiB-contiguous free pages, so the huge mapping (which needs them) silently falls back to 4 KiB PTEs. We initially mis-attributed this to a kernel config (`CONFIG_READ_ONLY_THP_FOR_FS` unset); the reproduced run **disproves** that — file PMD mapping works here (`FilePmdMapped = 64 MiB`). A **reboot** defragmented memory and the gap appeared on the very first read (+9.4 %). Lesson: THP-based effects depend on the *runtime memory state*, not just the code — a long-running, fragmented system can hide them entirely.

Evidence index

File	Evidence
<code>task2/p10_reboot_confirmed.txt</code>	gap reproduced (+9.4 %); dTLB 8.79M→18K; FilePmdMapped v2=64MiB, v1=0; LLC equal
<code>task2/p5_folio_hist_{v1,v2}.txt</code>	folio-order histogram: $v2 = 32 \times 2$ MiB PMD folios; $v1 = 16$ KiB
<code>task2/p1b_filefrag.txt</code> , <code>p2_perfstat_*</code>	fragmentation ruled out (zero major faults / disk I/O)
<code>task2/p5c_force_thp.txt</code>	independent dTLB-lever magnitude (<code>hugetlb</code>): +12.7–20.2 %
<code>task2/p4_page_cache_ra_order.txt</code> , <code>p4_do_set_pmd.txt</code>	kernel source: folio order + PMD map
<code>task2/p8_*</code> , <code>p9_recheck.txt</code>	long-uptime VM: gap absent, all counters flat (the §7 detour)

Appendix A — Commands Executed (Runbooks)

Every command run in this investigation, organised by phase. Outputs are in Appendix B.

`task1/RUNBOOK.md`

Task 1 Runbook – Pipe latency improves under background CPU load

```
> Run everything on `course-08` under a session log:
> `script -f ~/hw1/logs/task1_$(date +%Y%m%d_%H%M).log`
> Save each tool's raw output as its own file in this dir (named by what it tested).
> **PREDICT in writeup/notes.md before every measured run.** No peeking at results first.
```

```
`LP=/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe`
```

```
---
```

Phase 0 – Environment as shipped (capture BEFORE changing anything)

```
...
```

```
uname -a
systemd-detect-virt          # are we a KVM guest? changes how C-states appear
nproc; lscpu
cat /proc/cpuinfo | grep -E 'model name|MHz' | head
cpupower frequency-info      # governor + driver (intel_pstate? acpi-cpufreq?)
cpupower idle-info           # which C-states exist + exit latencies
cat /sys/devices/system/cpu/cpu0/cpuidle/state*/name 2>/dev/null
cat /proc/sys/kernel/perf_event_paranoid
ls -la ~                     # homework binaries/sources
...
```

Save as `task1/env\_asshipped.txt`. → fill the Environment block in notes.md.

Phase 1 – Baseline + variance (establish the effect AND the noise floor)

```
...
```

```
# no load, n=10
for i in $(seq 10); do $LP; done | tee baseline_noload.txt
# with load, n=10 (3 hogs on a 4-core box)
for i in $(seq 10); do stress-ng --cpu 3 --timeout 15s >/dev/null 2>&1 & sleep 1; $LP; wait;
done | tee baseline_load.txt
...
```

Report median / min / spread for each. Effect must be >> run-to-run noise before continuing.

Phase 2 – Discriminating probes (the three competing mechanisms)

```
> NOTE: This is a KVM guest with NO cpufreq driver and NO cpuidle states (see
> env_asshipped.txt). The old cpupower knobs below do NOT exist. We probe indirectly.
> Capture EVERYTHING with `2>&1 | tee` – lat_pipe prints to stderr.
```

```
`LP=/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe`
```

One `perf stat` decomposes both suspects at once:

- **F** (host frequency): the `GHz` reading (cycles / task-clock). If load vs no-load GHz is ~equal, frequency did not change → F is OUT.
- **I** (HLT idle / wakeup wait): the gap between wall time and CPU time – perf's `seconds time elapsed` vs `task-clock` / `CPUs utilized`. Lots of wall time with little CPU time = the process spent the round-trip waiting (the wakeup/halt cost). If that wait shrinks under load, I is IN.

Probe 1 – perf stat decomposition (NO load)

```
...
```

```
perf stat -e task-clock,cycles,instructions,ref-cycles,context-switches,cpu-migrations \
$LP 2>&1 | tee task1/p2_perfstat_noload.txt
...
```

Probe 2 – perf stat decomposition (WITH load)

```
...
```

```
stress-ng --cpu 3 --timeout 30s >/dev/null 2>&1 & sleep 1
perf stat -e task-clock,cycles,instructions,ref-cycles,context-switches,cpu-migrations \
$LP 2>&1 | tee task1/p2_perfstat_load.txt
wait
...
```

Probe 3 – pin BOTH pipe ends to one core, NO load (core can never go idle)

```
...
```

```
for i in $(seq 5); do taskset -c 0 $LP; done 2>&1 | tee task1/p2_pin1core_noload.txt
```

...

Interpretation (for the analyst, not the runner): if pin-to-one-core reproduces the ~5 μ s speedup with no bg load, idle-removal is sufficient – but it also makes the core 100% busy, so the host may raise frequency too; Probe 1/2's GHz reading disambiguates which it was.

Then `./scripts/vmsync "task1 p2 perfstat + pin"`, report the three files, and STOP.
If the PMU is unavailable in the guest, ``cycles`/`instructions`` show `<not supported>` – report that; the software ``task-clock`` / `elapsed` / `CPUs-utilized` fields still decompose I.

Phase 2B – Isolate idle (I) from placement (P), + direct wakeup latency
``LP=/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe``

Probe 4 – cross-core, cores allowed to go IDLE (pipe on cpus 0,1, no load)

...

`for i in $(seq 5); do taskset -c 0,1 $LP; done 2>&1 | tee task1/p2b_pin2core_idle.txt`

...

Probe 5 – cross-core, cores kept BUSY (one spinner pinned to each of 0,1)

Same placement as Probe 4; the ONLY change is the cores never idle.

...

`taskset -c 0 stress-ng --cpu 1 --timeout 40s >/dev/null 2>&1 &`

`taskset -c 1 stress-ng --cpu 1 --timeout 40s >/dev/null 2>&1 &`

`sleep 1`

`for i in $(seq 5); do taskset -c 0,1 $LP; done 2>&1 | tee task1/p2b_pin2core_busy.txt`

`wait`

...

Decision: Probe4 slow (~11) + Probe5 fast (~5) $\Rightarrow$ idle is the cause, placement held constant
 $\Rightarrow$ ****P ruled out, I ruled in.**** If Probe5 is also ~11, idle is NOT sufficient and P is back in.

Probe 6 – direct wakeup latency + actual CPU placement (no load vs load)

...

no load:

`perf sched record -o p2b_sched_noload.data -- bash -c 'for i in $(seq 3); do "$LP"; done' 2>&1 | tail -2`

`perf sched latency -i p2b_sched_noload.data 2>&1 | tee task1/p2b_sched_latency_noload.txt`

`perf sched timehist -i p2b_sched_noload.data 2>&1 | head -40 | tee`

`task1/p2b_sched_timehist_noload.txt`

with load:

`stress-ng --cpu 3 --timeout 40s >/dev/null 2>&1 & sleep 1`

`perf sched record -o p2b_sched_load.data -- bash -c 'for i in $(seq 3); do "$LP"; done' 2>&1 | tail -2`

`perf sched latency -i p2b_sched_load.data 2>&1 | tee task1/p2b_sched_latency_load.txt`

`perf sched timehist -i p2b_sched_load.data 2>&1 | head -40 | tee`

`task1/p2b_sched_timehist_load.txt`

`wait`

...

This gives the wakeup-run delay (direct fingerprint of I) AND the CPU column shows whether the pipe ends are actually cross-core under load (tests the assumption, doesn't assume it).

Then `./scripts/vmsync "task1 p2b"`, report the files, STOP.

Phase 4 – Confirm P in kernel source + produce deck-sanctioned evidence

Winner is ****P (scheduler wake-placement)****. Two things the lecturer requires that we still owe:

(a) confirmation in ``/usr/src``, (b) a histogram and a sampled callgraph (assignment text; deck slides 32, 66, 69–70).

Probe 7 – locate the placement logic in `/usr/src` (mechanical; student interprets)

...

`ls /usr/src`

`F=$(ls -d /usr/src/linux-* 2>/dev/null | head -1); echo "src=$F"`

`grep -n 'select_idle_sibling\|wake_affine\|select_task_rq_fair' $F/kernel/sched/fair.c | head`

dump the two functions for reading:

`sed -n '/^static int\s*$/,/^}/p' /dev/null # (placeholder)`

`awk '/select_idle_sibling\(struct task_struct/{p=1} p{print} /^}/{if(p)exit}'`

`$F/kernel/sched/fair.c > task1/p4_select_idle_sibling.txt`

`awk '/select_task_rq_fair\(/{p=1} p{print} /^}/{if(p)exit}'`

`$F/kernel/sched/fair.c > task1/p4_select_task_rq_fair.txt`

`wc -l task1/p4_select_*.txt`

```

...
(If awk ranges miss, just `grep -n` the function start lines and `sed -n 'START,+120p'` to
dump.)

### Probe 8 – sampled callgraph of the wakeup/switch path (deck's scheduler recipe, slide 69)
...
LP=/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe
sudo perf record -e sched:sched_switch -g -o task1/p4_cs_noload.data -- \
  bash -c 'for i in 1 2 3; do "$LP"; done' 2>&1 | tail -2
sudo perf report -i task1/p4_cs_noload.data --stdio 2>/dev/null | head -80 \
  | tee task1/p4_callgraph_noload.txt
sudo chown ubuntu task1/p4_cs_noload.data task1/p4_callgraph_noload.txt
# keep the .data only if small (<50M); else delete after capturing the report.
...

We want to SEE the no-load wakeup go through `try_to_wake_up → select_task_rq → ttwu_queue`
(remote/cross-CPU), i.e. the placement machinery, in the callgraph.

### Probe 9 – off-CPU latency histogram, no-load vs load (the assignment's "histogram", slide
32)
Off-CPU time per pipe block = sched_switch(out, blocked) → sched_switch(in). It contains the
wakeup cost; its distribution should shift between conditions.
...

BT='tracepoint:sched:sched_switch /args->prev_comm=="lat_pipe" && args->prev_state/ { @t[args->
prev_pid]=nsecs; }
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }'
# no load (run ~8s while lat_pipe runs in another shell, or wrap):
sudo bpftrace -e "$BT" -c "$LP" 2>&1 | tee task1/p4_offcpu_hist_noload.txt
# with load:
stress-ng --cpu 3 --timeout 20s >/dev/null 2>&1 & sleep 1
sudo bpftrace -e "$BT" -c "$LP" 2>&1 | tee task1/p4_offcpu_hist_load.txt
wait
...

Then `./scripts/vmsync "task1 p4 source + callgraph + histogram"`, report the files, STOP.
Note for writeup: governor/idle pinning (deck slide 65) is N/A here – the guest exposes no
cpufreq/cpuidle; we instead confirmed frequency was constant via cycles/ref-cycles.

```

```
# Task 2 Runbook – v2 (4 MiB prepare) random-read ~8–9% faster

> Run on `course-08` under: `script -f ~/hw1/logs/task2_$(date +%Y%m%d_%H%M).log`
> **PREDICT in notes.md before every measured run.**

...

COMMON="--file-num=1 --file-total-size=64M"
RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run"
...

## Phase 0 – Environment (capture once) → task2/p0_env.txt
...
{ uname -a; echo; free -h; echo; systemd-detect-virt; echo;
  mount | grep -E "$(stat -c %m ~)"; echo;          # which fs is $HOME on? (ext4? xfs?)
  echo "THP:"; cat /sys/kernel/mm/transparent_hugepage/enabled;
  echo "sysbench:"; sysbench --version; } 2>&1 | tee task2/p0_env.txt
...

## Phase 1 – Setup + baseline (confirm the gap is real and stable) → task2/p1_baseline_gap.txt
...
mkdir -p ~/v1 ~/v2
( cd ~/v1 && sysbench fileio $COMMON prepare )
( cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M prepare )
# n>=5 each, interleaved, to separate gap from run-to-run noise:
for i in $(seq 5); do
  echo "--- iter $i v1 ---"; ( cd ~/v1 && $RUN ) | grep -E 'reads/s|read, MiB';
  echo "--- iter $i v2 ---"; ( cd ~/v2 && $RUN ) | grep -E 'reads/s|read, MiB';
done 2>&1 | tee task2/p1_baseline_gap.txt
...

Gap must be stable and > noise. Note: everything that differs was baked in at PREPARE time.

## Phase 1B – Diagnose the MISSING gap (the gap did not reproduce under THP=always)
The Phase 1 baseline showed v1 ≈ v2 (no 8–9% gap). Before anything else, find out why.

### Sanity – were the two files actually prepared differently?
...
filefrag ~/v1/test_file.0 ~/v2/test_file.0 2>&1 | tee task2/p1b_filefrag.txt
...
(Expected v1 to have many more extents than v2. Confirms the prepare block-size difference took.)

### Decisive – sweep THP and see if the gap appears (toggle-the-effect)
Suspect: THP=always gives large folios to BOTH files, erasing the write-size difference.
For each THP mode, drop caches, re-prepare under that mode, re-run:
...
COMMON="--file-num=1 --file-total-size=64M"
RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run"
for thp in always madvice never; do
  echo $thp | sudo tee /sys/kernel/mm/transparent_hugepage/enabled >/dev/null
  sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
  rm -rf ~/v1 ~/v2; mkdir -p ~/v1 ~/v2
  ( cd ~/v1 && sysbench fileio $COMMON prepare ) >/dev/null 2>&1
  ( cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M prepare ) >/dev/null 2>&1
  echo "=== THP=$thp ==="
  for i in 1 2 3; do
    printf 'v1 '; ( cd ~/v1 && $RUN ) | grep -oP 'reads/s:\s*\K[0-9.]+'
    printf 'v2 '; ( cd ~/v2 && $RUN ) | grep -oP 'reads/s:\s*\K[0-9.]+'
  done
done 2>&1 | tee task2/p1b_thp_sweep.txt
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled >/dev/null # restore as-shipped
...

Read: does a v2>v1 gap (~8–9%) appear under `madvice` and/or `never` but not `always`?
That localizes the effect to THP/folio size. If NO gap appears under any mode, the mechanism
is something else and we keep digging.
```

Phase 2 – Diagnose with the LECTURER'S technique (caching deck slides 58, 64–65)
His Redis example: headline counter = **page-faults**; then **count huge-page usage** on the fault path with bpftrace ("count, don't eyeball"). We apply the same to the file page cache.

2a – perf stat, page-faults FIRST (his slide 58 comparison)

```
...  
( cd ~/v1 && perf stat -e page-faults,minor-faults,dTLB-load-misses,cycles,instructions $RUN )  
2>&1 | tee task2/p2_perfstat_v1.txt  
( cd ~/v2 && perf stat -e page-faults,minor-faults,dTLB-load-misses,cycles,instructions $RUN )  
2>&1 | tee task2/p2_perfstat_v2.txt  
...
```

Read: does v2 have FEWER page-faults than v1? (instructions ~equal.) Equal page-faults ⇒ consistent with the missing gap ⇒ go to 2b to learn whether huge pages are used for both/neither.

2b – Are huge pages / large folios actually used? (his VM_FAULT_FALLBACK count, slides 64–65)

Direct view – huge file mappings while each run is in flight:

```
...  
( cd ~/v1 && $RUN ) & sleep 2; grep -E 'File|Pmd|Huge|Rss' /proc/$(pgrep -n sysbench)/smaps_rollback | tee task2/p2_smaps_v1.txt; wait  
( cd ~/v2 && $RUN ) & sleep 2; grep -E 'File|Pmd|Huge|Rss' /proc/$(pgrep -n sysbench)/smaps_rollback | tee task2/p2_smaps_v2.txt; wait  
...
```

Count the PMD (huge) vs base-page file faults on the read path (analogous to his fallback count):

```
...  
BT='kprobe:do_set_pmd { @pmd_huge = count(); } kprobe:set_pte_range { @base = count(); }'  
( cd ~/v1 && $RUN ) & sleep 1; sudo bpftrace -e "$BT" -c "sleep 3" 2>&1 | tee  
task2/p2_faultcount_v1.txt; wait  
( cd ~/v2 && $RUN ) & sleep 1; sudo bpftrace -e "$BT" -c "sleep 3" 2>&1 | tee  
task2/p2_faultcount_v2.txt; wait  
...
```

(If `do_set_pmd`/`set_pte_range` aren't probeable on this kernel, fall back to counting `filemap_fault` and tracing folio order; report what names exist via `grep`.)

PREDICT first: are huge pages used for v2 but not v1, or for both, or neither? That answers WHY the gap is (or isn't) present.

Phase 3 – Rule out the seductive wrong answer: disk fragmentation

v1's small writes fragment the file; v2's 4M writes don't. But is the run even touching disk?

```
...  
filefrag -v ~/v1/test_file.0 | tail -3          # extent count v1  
filefrag -v ~/v2/test_file.0 | tail -3          # extent count v2 (expect far fewer)  
# Are both fully in page cache during the run? If yes, disk layout is irrelevant:  
vmtouch ~/v1/test_file.0 ~/v2/test_file.0      2>/dev/null || \  
( cd ~/v1 && perf stat -e block:block_rq_issue $RUN ) # ~0 block I/O => cache-resident  
...
```

Write the explicit ruling-out in notes.md WITH the data (extent counts + zero block I/O).

Phase 3b – Confirm the real mechanism: folio order in the page cache

The persistent prepare-time difference is the size of the folios the page cache built.

```
...  
# folio sizes mapped for the file (look for FilePmdMapped / large mappings):  
grep -E 'File|Anon|Pmd' /proc/$(pgrep -n sysbench)/smaps_rollback 2>/dev/null # during a run  
# or trace folio allocation order during prepare:  
sudo bpftrace -e 'kprobe:__filemap_add_folio { @order = hist(arg3); }' # adjust arg per  
kernel  
# (run a v1 prepare vs a v2 prepare under this; compare the order histograms)  
...
```

Phase 3 – Confirm folio order, find the code, try to reproduce the gap

Phase 2 showed: v2 builds larger folios (2× fewer faults) but both stay sub-PMD ⇒ no huge mapping ⇒ no dTLB win ⇒ no throughput gap. Now: prove the folio sizes, locate the kernel logic, and try to push folios to PMD so the gap appears (toggle-the-effect, like `MALLOC_TOP_PAD`).

```

### 3a – Confirm folio order built during prepare (bpftrace)
...
BT='tracepoint:filemap:mm_filemap_add_to_page_cache { @folios = count(); }'
COMMON="--file-num=1 --file-total-size=64M"
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sudo bpftrace -e "$BT" -c "bash -lc 'cd ~/v1 && sysbench fileio $COMMON prepare'" 2>&1 | tee
task2/p3_folios_v1.txt
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sudo bpftrace -e "$BT" -c "bash -lc 'cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M
prepare'" 2>&1 | tee task2/p3_folios_v2.txt
...

```

16384 pages / @folios = avg pages per folio. Expect v2 > v1, both < 512 (PMD).

```

### 3b – Reproduction attempts (make v2 reach PMD ⇒ make the gap appear)
...
RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --
time=5 run"
# (i) THP sweep – does any mode change FilePmdMapped / the gap? (the deferred sweep)
for thp in always madvise never; do
    echo $thp | sudo tee /sys/kernel/mm/transparent_hugepage/enabled >/dev/null
    sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
    rm -rf ~/v1 ~/v2; mkdir -p ~/v1 ~/v2
    ( cd ~/v1 && sysbench fileio $COMMON prepare ) >/dev/null 2>&1
    ( cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M prepare ) >/dev/null 2>&1
    echo "=== THP=$thp ==="; for i in 1 2 3; do
        printf 'v1 '; ( cd ~/v1 && $RUN ) | grep -oP 'reads/s:\s*\K[0-9.]+';
        printf 'v2 '; ( cd ~/v2 && $RUN ) | grep -oP 'reads/s:\s*\K[0-9.]+'; done
done 2>&1 | tee task2/p3_thp_sweep.txt
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled >/dev/null
# (ii) bigger working set – larger files may reach higher-order/PMD folios
BIG="--file-num=1 --file-total-size=1G"
BRUN="sysbench fileio $BIG --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --
time=5 run"
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
rm -rf ~/b1 ~/b2; mkdir -p ~/b1 ~/b2
( cd ~/b1 && sysbench fileio $BIG prepare ) >/dev/null 2>&1
( cd ~/b2 && sysbench fileio $BIG --file-block-size=4M prepare ) >/dev/null 2>&1
{ echo "=== 1G v1 ==="; ( cd ~/b1 && $BRUN ) | grep -E 'reads/s|read, MiB';
  ( cd ~/b1 && perf stat -e page-faults,dTLB-load-misses $BRUN ) 2>&1 | grep -E 'page-
faults|dTLB';
  echo "=== 1G v2 ==="; ( cd ~/b2 && $BRUN ) | grep -E 'reads/s|read, MiB';
  ( cd ~/b2 && perf stat -e page-faults,dTLB-load-misses $BRUN ) 2>&1 | grep -E 'page-
faults|dTLB';
} 2>&1 | tee task2/p3_bigfile.txt
( cd ~/b1 && sysbench fileio $BIG cleanup ); ( cd ~/b2 && sysbench fileio $BIG cleanup )
...

```

Read: does a v2>v1 reads/s gap appear under any THP mode or at 1 GiB, with FilePmdMapped>0 / fewer dTLB-misses? If yes → gap reproduced and tied to huge mapping. If never → gap is genuinely dormant in this kernel; we explain it via the sub-PMD folio cap + source.

Phase 4 – Kernel-source confirmation (/usr/src): folio-order cap + PMD condition
Explain (a) why v2's writes build larger folios, (b) why they stay sub-PMD, (c) why no huge mapping.

```

F=$(ls -d /usr/src/linux-* 2>/dev/null | head -1); echo "src=$F"
# (a/b) folio order selection + cap:
grep -n 'page_cache_ra_order\|MAX_PAGECACHE_ORDER\|mapping_max_folio' $F/mm/readahead.c
$F/mm/filemap.c $F/include/linux/pagemap.h | head
awk '/page_cache_ra_order/{p=1} p{print} /^}/{if(p)exit}' $F/mm/readahead.c >
task2/p4_page_cache_ra_order.txt
# (c) the PMD (huge) file-map path + its gate:
grep -n 'do_set_pmd' $F/mm/memory.c $F/mm/filemap.c | head
awk '/^vm_fault_t do_set_pmd\(|do_set_pmd\(struct/{p=1} p{print} /^}/{if(p)exit}' $F/mm/memory.c
> task2/p4_do_set_pmd.txt
# (c) DECISIVE config check – is file-backed THP even compiled in?

```

```

zcat /proc/config.gz 2>/dev/null | grep -E 'READ_ONLY_THP_FOR_FS|TRANSPARENT_HUGEPAGE' || \
    grep -E 'READ_ONLY_THP_FOR_FS|TRANSPARENT_HUGEPAGE' /boot/config-$(uname -r) | tee
task2/p4_kconfig_thp.txt
wc -l task2/p4_*.txt
```

```

Connect to the data: `page\_cache\_ra\_order` decides folio order from the readahead/write size (why v2's 4M writes → larger folios) and caps it (why ≤ a few pages here); `do\_set\_pmd` only installs a huge PMD mapping when the folio is PMD-order AND aligned (why FilePmdMapped=0 with sub-PMD folios); if `CONFIG\_READ\_ONLY\_THP\_FOR\_FS` is **not** set, mmap'd ext4 reads can **never** get PMD file mappings → the dTLB-driven gap is structurally impossible on this kernel. **Student makes the final connection.**

**## Phase 5 – Prove the mechanism (don't just assert it) + the histogram**  
 Closes two audit gaps: (a) a real histogram, (b) confirm the dTLB/huge-page lever is actually worth the gap on this hardware – the lecturer's "toggle the effect / measure again" (intro s.37).

**### 5a – Folio-order histogram during prepare (the "histogram" data form)**  
 ```

```

COMMON="--file-num=1 --file-total-size=64M"
# find the right function/arg first:
sudo bpftrace -l 'kprobe:filemap_alloc_folio' ; sudo bpftrace -l 'tracepoint:filemap:*'
# folio order distribution (arg1 of filemap_alloc_folio is the order; verify & adjust):
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sudo bpftrace -e 'kprobe:filemap_alloc_folio { @order_v1 = lhist(arg1,0,12,1); }' \
    -c "bash -lc 'cd ~/v1 && sysbench fileio $COMMON prepare'" 2>&1 | tee
task2/p5_folio_hist_v1.txt
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sudo bpftrace -e 'kprobe:filemap_alloc_folio { @order_v2 = lhist(arg1,0,12,1); }' \
    -c "bash -lc 'cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M prepare'" 2>&1 | tee
task2/p5_folio_hist_v2.txt
```

```

(If `filemap\_alloc\_folio` isn't probeable or arg1 isn't the order, report what `bpftrace -lv` shows and use `\_\_filemap\_add\_folio`/`folio\_alloc` or the tracepoint's `order` field instead.)

**### 5b – DECISIVE: is the dTLB/huge-page lever actually worth the gap here?**  
 Same random 4 KiB reads over an anon mapping, THP ON vs OFF, measured by dTLB-misses + throughput.  
 ```

```

cat > /tmp/thp_rndrd.c <<'EOF'
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>
#include <sys/mman.h>
int main(int argc, char **argv){
    size_t MB = argc>1?atol(argv[1]):64; int huge = argc>2?atoi(argv[2]):0;
    size_t sz = MB<<20;
    char *p = mmap(NULL,sz,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
    if(p==MAP_FAILED){perror("mmap");return 1;}
    madvise(p,sz, huge?MADV_HUGEPAGE:MADV_NOHUGEPAGE);
    memset(p,1,sz); /* fault in (collapse to THP if huge) */
    size_t pages = sz/4096; uint64_t s=0,r=0x9e3779b97f4a7c15ULL;
    size_t iters=200000000;
    struct timespec a,b; clock_gettime(CLOCK_MONOTONIC,&a);
    for(size_t i=0;i<iters;i++){ r^=r<<13; r^=r>>7; r^=r<<17; s+=p[(r%pages)*4096]; }
    clock_gettime(CLOCK_MONOTONIC,&b);
    double t=(b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
    fprintf(stderr,"MB=%zu huge=%d reads/s=%.0f sum=%lu\n",MB,huge,iters/t,(unsigned long)s);
}
EOF

```



```

    return 0;
}
EOF
gcc -O2 -o /tmp/thp_rndrd /tmp/thp_rndrd.c
{ for MB in 64 1024; do for h in 0 1; do
    echo "=== ${MB}MB huge=$h ==="
    sudo perf stat -e dTLB-load-misses,cycles,instructions /tmp/thp_rndrd $MB $h
done; done; } 2>&1 | tee task2/p5_thp_dtlb_test.txt
# sanity: confirm THP actually mapped (AnonHugePages>0 for huge=1) – quick check:
grep -H AnonHugePages /proc/self/smaps_rollup 2>/dev/null | tee -a task2/p5_thp_dtlb_test.txt
...

Read: for huge=1 vs huge=0, do dTLB-load-misses drop sharply AND reads/s rise? If the rise is
~the 8–9% gap (and bigger at 1 GiB) ⇒ mechanism PROVEN: the gap is the huge-page dTLB win that
file-THP would deliver but `CONFIG_READ_ONLY_THP_FOR_FS=off` blocks. If no benefit ⇒ our dTLB
story is WRONG – report it, we rethink (maybe LLC/contiguity; also capture LLC-load-misses
then).
Then `./scripts/vmsync "task2 p5 folio hist + THP dTLB proof"`, report files, STOP.

## Phase 5c – FORCE huge mapping and measure the dTLB win (fixes 5b)
5b's MADV_HUGEPAGE never engaged THP. This version FORCES it three ways and self-reports whether
it worked, so the dTLB contrast is real. Working set > LLC (256M/1G) to isolate TLB from cache.
...

cat > /tmp/thp_rndrd2.c <<'EOF'
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>
#include <errno.h>
#include <sys/mman.h>
#ifdef MADV_COLLAPSE
#define MADV_COLLAPSE 25
#endif
static long anonhuge_kb(void){
    FILE*f=fopen("/proc/self/smaps_rollup","r"); if(!f) return -1;
    char l[256]; long kb=-1;
    while(fgets(l,sizeof l,f)) if(sscanf(l,"AnonHugePages: %ld kB",&kb)==1) break;
    fclose(f); return kb;
}
int main(int argc,char**argv){
    size_t MB=argc>1?atol(argv[1]):256; int mode=argc>2?atoi(argv[2]):0; /* 0=4K 1=COLLAPSE
2=HUGETLB */
    size_t sz=MB<<20; int flags=MAP_PRIVATE|MAP_ANONYMOUS; if(mode==2) flags|=MAP_HUGETLB;
    char*p=mmap(NULL,sz,PROT_READ|PROT_WRITE,flags,-1,0);
    if(p==MAP_FAILED){ fprintf(stderr,"mmap(mode=%d) failed: %s\n",mode,strerror(errno)); return
1; }
    memset(p,1,sz);
    if(mode==1 && madvise(p,sz,MADV_COLLAPSE)!=0) fprintf(stderr,"MADV_COLLAPSE failed:
%s\n",strerror(errno));
    fprintf(stderr,"mode=%d MB=%zu AnonHugePages_kb=%ld%s\n",mode,MB,anonhuge_kb(),mode=="
(hugetlb: see HugePages_Free)": "");
    size_t pages=sz/4096; uint64_t s=0,r=0x9e3779b97f4a7c15ULL; size_t iters=200000000;
    struct timespec a,b; clock_gettime(CLOCK_MONOTONIC,&a);
    for(size_t i=0;i<iters;i++){ r^=r<<13; r^=r>>7; r^=r<<17; s+=p[(r%pages)*4096]; }
    clock_gettime(CLOCK_MONOTONIC,&b);
    double t=(b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
    fprintf(stderr,"mode=%d MB=%zu reads/s=%.0f sum=%lu\n",mode,MB,iters/t,(unsigned long)s);
    return 0;
}
EOF
gcc -O2 -o /tmp/thp_rndrd2 /tmp/thp_rndrd2.c
# base (4K) vs MADV_COLLAPSE (2M THP) – watch AnonHugePages_kB in the output (must be >0 for
mode 1):
{ for MB in 256 1024; do for m in 0 1; do
    echo "=== ${MB}MB mode=$m ==="

```

```

    sudo perf stat -e dTLB-load-misses,cycles,instructions /tmp/thp_rndrd2 $MB $m
done; done; } 2>&1 | tee task2/p5c_force_thp.txt
# guaranteed huge via hugetlb - reserve, run, release:
echo 600 | sudo tee /proc/sys/vm/nr_hugepages >/dev/null
grep -E 'HugePages_(Total|Free)' /proc/meminfo | tee -a task2/p5c_force_thp.txt
{ for MB in 256 1024; do
    echo "=== ${MB}MB mode=2 (MAP_HUGETLB) ==="
    sudo perf stat -e dTLB-load-misses,cycles,instructions /tmp/thp_rndrd2 $MB 2
done; } 2>&1 | tee -a task2/p5c_force_thp.txt
echo 0 | sudo tee /proc/sys/vm/nr_hugepages >/dev/null # release as-shipped
...

Read: for the runs where huge mapping ENGAGED (AnonHugePages_kB>0 for mode 1, or HugePages used
for
mode 2), do dTLB-load-misses drop sharply vs mode 0, and reads/s rise? The size of that rise is
the
dTLB-win magnitude on this CPU - the number proving the Phase-4/5 claim. Report AnonHugePages_kB
and
HugePages_Free so we KNOW which runs actually got huge pages. Then vmsync "task2 p5c", report,
STOP.

## Phase 7 - Reproduce the mechanism on a REAL file mapping (tmpfs/shmem huge pages)
ext4 file-THP is compiled out (READ_ONLY_THP_FOR_FS unset), but tmpfs/shmem huge pages are a
SEPARATE path with a runtime knob - NOT compiled out. Use it to get a genuine PMD-mapped *file*
mmap (upgrades the $5b anonymous hugetlb proxy to an actual file). Honest scope: this reproduces
the
folio-PMD-map->dTLB MECHANISM on a file, NOT the literal ext4 v1/v2 gap.
...

COMMON="--file-num=1 --file-total-size=64M"
RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --
time=5 run"
cat /sys/kernel/mm/transparent_hugepage/shmem_enabled | tee task2/p7_shmem_env.txt
sudo mkdir -p /mnt/hugetmp
...

### 7a - tmpfs huge=always (PMD file mapping should engage)
...

sudo mount -t tmpfs -o huge=always,size=2G tmpfs /mnt/hugetmp
sudo chown $USER /mnt/hugetmp; mkdir -p /mnt/hugetmp/v2
( cd /mnt/hugetmp/v2 && sysbench fileio $COMMON --file-block-size=4M prepare ) >/dev/null 2>&1
( cd /mnt/hugetmp/v2 && $RUN ) & sleep 2
grep -E 'Pmd|Huge|Rss|Anon' /proc/$(pgrep -n sysbench)/smaps_rollup ; wait
( cd /mnt/hugetmp/v2 && perf stat -e dTLB-load-misses,cycles,instructions $RUN ) 2>&1
...

all of the above -> `2>&1 | tee task2/p7_shmem_huge.txt`
### 7b - same tmpfs, huge=never (clean on/off toggle)
...

sudo umount /mnt/hugetmp
sudo mount -t tmpfs -o huge=never,size=2G tmpfs /mnt/hugetmp
sudo chown $USER /mnt/hugetmp; mkdir -p /mnt/hugetmp/v2
( cd /mnt/hugetmp/v2 && sysbench fileio $COMMON --file-block-size=4M prepare ) >/dev/null 2>&1
( cd /mnt/hugetmp/v2 && $RUN ) & sleep 2
grep -E 'Pmd|Huge|Rss|Anon' /proc/$(pgrep -n sysbench)/smaps_rollup ; wait
( cd /mnt/hugetmp/v2 && perf stat -e dTLB-load-misses,cycles,instructions $RUN ) 2>&1
...

all of the above -> `2>&1 | tee task2/p7_shmem_base.txt`, then `sudo umount /mnt/hugetmp`.
PREDICT FIRST (ShmemPmdMapped, dTLB-misses, reads/s for each). Discriminating read:
- huge=always: ShmemPmdMapped>0, dTLB-misses collapse ~8.7M->thousands, reads/s rise -> MECHANISM
reproduced on a file mapping. (Do NOT relabel as "the ext4 gap reproduced".)
- huge=always ALSO shows no PMD -> second data point that THP is broken on this image
(MADV_COLLAPSE gave EINVAL, anon THP refused at [always]) -> strengthens the wrong-image
argument.
Then `./scripts/vmsync "task2 p7 shmem file-THP"`, report, STOP.

## Phase 8 - Back to basics: clean EXACT reproduction + FULL counters (incl. LLC)
The gap reproduces for others on this VM -> we erred. Re-run the assignment verbatim on an IDLE
system and capture the counter we skipped (LLC) to find what really differs.

```

```

### 8a – is the VM busy? (test the "background load suppressed it" idea)
```
{ uptime; echo; ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head -12; echo;
 cat /proc/sys/vm/transparent_hugepage/enabled 2>/dev/null;
 grep -E 'MemFree|MemAvailable|AnonHugePages' /proc/meminfo; } 2>&1 | tee task2/p8_vm_state.txt
```

If anything heavy is running (node/bun/claude/sysbench), pause/stop it for the measurement if you can.

### 8b – EXACT assignment sequence, fresh, with FULL counters
```
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
rm -rf ~/v1 ~/v2; mkdir -p ~/v1 ~/v2
COMMON="--file-num=1 --file-total-size=64M"
(cd ~/v1 && sysbench fileio $COMMON prepare) >/dev/null
(cd ~/v2 && sysbench fileio $COMMON --file-block-size=4M prepare) >/dev/null
RUN="sysbench fileio $COMMON --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run"
EV="cycles,instructions,dTLB-load-misses,LLC-loads,LLC-load-misses,L1-dcache-load-misses"
run v1 THEN v2 (assignment order), 3 reps, FULL counters:
for rep in 1 2 3; do
 echo "=== rep $rep v1 ==="; (cd ~/v1 && perf stat -e $EV $RUN) 2>&1 | grep -E
'reads/s|cycles|instructions|dTLB|LLC|L1-dcache|elapsed'
 echo "=== rep $rep v2 ==="; (cd ~/v2 && perf stat -e $EV $RUN) 2>&1 | grep -E
'reads/s|cycles|instructions|dTLB|LLC|L1-dcache|elapsed'
done 2>&1 | tee task2/p8_clean_fullcounters.txt
FilePmdMapped on a clean v2 run (did our cache-thrashing earlier suppress it?):
(cd ~/v2 && $RUN) & sleep 2; grep -E 'Pmd|Huge|Anon|Rss' /proc/$(pgrep -n
sysbench)/smaps_rollback 2>&1 | tee -a task2/p8_clean_fullcounters.txt; wait
```

Read: does v2 show ~8% higher reads/s now? If so, WHICH counter differs – **LLC-load-misses**? dTLB? cycles? That counter is the real mechanism. If still flat, report the full counters anyway so we can see where v1 and v2 actually diverge (or confirm they don't). Then vmsync, report, STOP.

## Cleanup
```
(cd ~/v1 && sysbench fileio $COMMON cleanup); (cd ~/v2 && sysbench fileio $COMMON cleanup)
```

```

Appendix B — Raw Logs and Tool Outputs

Task 1 — raw output

logs/task1_20260610_1427.log

```
Script started on 2026-06-10 14:27:48+00:00 [COMMAND="bash /tmp/task1_p01.sh" <not executed on
terminal>]
##### PHASE 0 — environment as shipped #####
wrote task1/env_asshipped.txt
##### PHASE 1 — baseline no-load (n=10) #####
# Task1 Phase 1 — lat_pipe, NO load, n=10 — 2026-06-10T14:27:48+00:00
Pipe latency: 11.2042 microseconds
Pipe latency: 10.4985 microseconds
Pipe latency: 11.1315 microseconds
Pipe latency: 11.1770 microseconds
Pipe latency: 11.1627 microseconds
Pipe latency: 11.1287 microseconds
Pipe latency: 11.1157 microseconds
Pipe latency: 11.0622 microseconds
Pipe latency: 10.5657 microseconds
Pipe latency: 9.8751 microseconds
##### PHASE 1 — baseline with load: stress-ng --cpu 3 (n=10) #####
# Task1 Phase 1 — lat_pipe, WITH load (stress-ng --cpu 3), n=10 — 2026-06-10T14:28:04+00:00
Pipe latency: 5.2355 microseconds
Pipe latency: 5.6613 microseconds
Pipe latency: 5.2755 microseconds
Pipe latency: 6.3949 microseconds
Pipe latency: 5.5291 microseconds
Pipe latency: 5.5314 microseconds
Pipe latency: 4.7368 microseconds
Pipe latency: 4.9441 microseconds
Pipe latency: 5.0993 microseconds
Pipe latency: 4.8185 microseconds
##### DONE #####

Script done on 2026-06-10 14:30:34+00:00 [COMMAND_EXIT_CODE="0"]
```

task1/baseline_load.txt

```
# Task1 Phase 1 — lat_pipe, WITH load (stress-ng --cpu 3), n=10 — 2026-06-10T14:28:04+00:00
```

task1/baseline_noload.txt

```
# Task1 Phase 1 — lat_pipe, NO load, n=10 — 2026-06-10T14:27:48+00:00
```

task1/env_asshipped.txt

Task1 Phase 0 – environment as shipped – 2026-06-10T14:27:48+00:00

=== uname -a ===

Linux course-08 7.0.0-15-generic #15-Ubuntu SMP PREEMPT_DYNAMIC Wed Apr 22 16:06:43 UTC 2026
x86_64 GNU/Linux

=== systemd-detect-virt ===

kvm

=== nproc ===

4

=== lscpu ===

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Address sizes:	46 bits physical, 57 bits virtual
Byte Order:	Little Endian
CPU(s):	4
On-line CPU(s) list:	0-3
Vendor ID:	GenuineIntel
Model name:	Intel(R) Xeon(R) Gold 5420+
CPU family:	6
Model:	143
Thread(s) per core:	1
Core(s) per socket:	1
Socket(s):	4
Stepping:	8
BogoMIPS:	4000.00
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclmulqdq dtes64 vmx ssse3 fma cx16 pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch cpuid_fault ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx_vnni avx512_bf16 wbnoinvd arat vnmi avx512vbmi umip pku ospke waitpkg avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq la57 rdpid bus_lock_detect cldemote movdiri movdir64b fsrm md_clear serialize tsxldtrk amx_bf16 avx512_fp16 amx_tile amx_int8 flush_l1d arch_capabilities
Virtualization:	VT-x
Hypervisor vendor:	KVM
Virtualization type:	full
L1d cache:	128 KiB (4 instances)
L1i cache:	128 KiB (4 instances)
L2 cache:	16 MiB (4 instances)
L3 cache:	64 MiB (4 instances)
NUMA node(s):	1
NUMA node0 CPU(s):	0-3
Vulnerability Gather data sampling:	Not affected
Vulnerability Ghostwrite:	Not affected
Vulnerability Indirect target selection:	Not affected
Vulnerability Itlb multihit:	Not affected
Vulnerability L1tf:	Not affected
Vulnerability Mds:	Not affected
Vulnerability Meltdown:	Not affected
Vulnerability Mmio stale data:	Not affected
Vulnerability Old microcode:	Not affected
Vulnerability Reg file data sampling:	Not affected
Vulnerability Retbleed:	Not affected
Vulnerability Spec rstack overflow:	Not affected
Vulnerability Spec store bypass:	Mitigation; Speculative Store Bypass disabled via prctl
Vulnerability Spectre v1:	Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2:	Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-IBRS SW sequence; BHI BHI_DIS_S
Vulnerability Srbds:	Not affected

```
Vulnerability Tsa:                Not affected
Vulnerability Tsx async abort:     Mitigation; TSX disabled
Vulnerability Vmscape:             Not affected
```

```
=== /proc/cpuinfo (model name|MHz) ===
model name : Intel(R) Xeon(R) Gold 5420+
cpu MHz    : 2000.000
model name : Intel(R) Xeon(R) Gold 5420+
cpu MHz    : 2000.000
model name : Intel(R) Xeon(R) Gold 5420+
cpu MHz    : 2000.000
model name : Intel(R) Xeon(R) Gold 5420+
cpu MHz    : 2000.000
```

```
=== cpupower frequency-info ===
analyzing CPU 3:
  no or unknown cpufreq driver is active on this CPU
  CPUs which run at the same hardware frequency: Not Available
  CPUs which need to have their frequency coordinated by software: Not Available
  maximum transition latency: Cannot determine or is not supported.
Not Available
  available cpufreq governors: Not Available
  Unable to determine current policy
  current CPU frequency: Unable to call to kernel
  boost state support:
    Supported: no
    Active: no
```

```
=== cpupower idle-info ===
CPUidle driver: none
CPUidle governor: menu
analyzing CPU 3:
```

```
CPU 3: No idle states
```

```
=== cpuidle state names ===
```

```
=== perf_event_paranoid ===
-1
```

```
=== ls -la ~ ===
```

```
total 96
drwxr-x--- 10 ubuntu ubuntu 4096 Jun 10 14:23 .
drwxr-xr-x  3 root  root   4096 May 22 15:07 ..
-rw-----  1 ubuntu ubuntu  376 Jun  9 18:56 .bash_history
-rw-r--r--  1 ubuntu ubuntu  220 Feb 13 12:16 .bash_logout
-rw-r--r--  1 ubuntu ubuntu 3771 Feb 13 12:16 .bashrc
drwx-----  3 ubuntu ubuntu 4096 Jun 10 13:42 .cache
drwxrwxr-x 10 ubuntu ubuntu 4096 Jun 10 14:27 .claude
-rw-----  1 ubuntu ubuntu 27468 Jun 10 14:23 .claude.json
drwxrwxr-x  3 ubuntu ubuntu 4096 Jun 10 14:22 .config
drwxrwxr-x  6 ubuntu ubuntu 4096 Jun 10 14:06 .git
-rw-rw-r--  1 ubuntu ubuntu  181 Jun 10 14:23 .gitconfig
drwxrwxr-x  3 ubuntu ubuntu 4096 Jun 10 13:43 .local
drwxrwxr-x  4 ubuntu ubuntu 4096 Jun 10 13:42 .npm
-rw-r--r--  1 ubuntu ubuntu  807 Feb 13 12:16 .profile
drwx-----  2 ubuntu ubuntu 4096 May 22 15:09 .ssh
drwxr-xr-x 10 ubuntu ubuntu 4096 Jun 10 14:16 hw1
-rw-r--r--  1 ubuntu ubuntu 4130 Jun  9 16:37 hw1_workspace.zip
```

task1/p2_perfstat_load.txt

Pipe latency: 69.1056 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

28749.27	msec task-clock
73484767995	cycles
26922915348	instructions
54567480720	ref-cycles
33982	context-switches
6	cpu-migrations

29.746661716 seconds time elapsed

27.719841000 seconds user

1.001958000 seconds sys

task1/p2_perfstat_noload.txt

Pipe latency: 69.6173 microseconds

Performance counter stats for '/usr/lib/lmbench/bin/x86_64-linux-gnu/lat_pipe':

1513.48	msec task-clock
1619968404	cycles
667048034	instructions
1202635280	ref-cycles
29252	context-switches
2	cpu-migrations

1.571171194 seconds time elapsed

0.587697000 seconds user

0.891950000 seconds sys

task1/p2_pin1core_noload.txt

Pipe latency: 5.4472 microseconds

Pipe latency: 5.3650 microseconds

Pipe latency: 5.4265 microseconds

Pipe latency: 5.4146 microseconds

Pipe latency: 5.4262 microseconds

task1/p2b_latpipe_cpudist_load.txt

531538 [0000]

284307 [0001]

654922 [0002]

73922 [0003]

task1/p2b_latpipe_cpudist_noload.txt

293394 [0000]

29002 [0001]

364682 [0002]

40576 [0003]

task1/p2b_pin2core_busy.txt

```
Pipe latency: 10.3362 microseconds  
Pipe latency: 10.9540 microseconds  
Pipe latency: 11.0637 microseconds  
Pipe latency: 11.0859 microseconds  
Pipe latency: 11.1015 microseconds
```

task1/p2b_pin2core_idle.txt

```
Pipe latency: 11.2613 microseconds  
Pipe latency: 11.3220 microseconds  
Pipe latency: 11.2336 microseconds  
Pipe latency: 11.2479 microseconds  
Pipe latency: 11.0964 microseconds
```

task1/p2b_sched_latency_load.txt

Task	Runtime ms	Count	Avg delay ms	Max delay ms	Max
delay start	Max delay end				
HeapHelper:(3)	26.603 ms	1487	avg: 0.790 ms	max: 6.020 ms	max
start: 1642313.655957 s	max end: 1642313.661977 s				
HTTP Client:75650	0.353 ms	7	avg: 0.474 ms	max: 3.223 ms	max
start: 1642314.487533 s	max end: 1642314.490756 s				
Bun Pool 3:75652	0.336 ms	19	avg: 0.208 ms	max: 3.776 ms	max
start: 1642309.713979 s	max end: 1642309.717755 s				
JITWorker:76568	46.253 ms	537	avg: 0.153 ms	max: 3.840 ms	max
start: 1642315.742914 s	max end: 1642315.746754 s				
Bun Pool 1:75649	0.258 ms	18	avg: 0.118 ms	max: 1.913 ms	max
start: 1642308.502840 s	max end: 1642308.504753 s				
claude:(2)	496.095 ms	5755	avg: 0.114 ms	max: 4.083 ms	max
start: 1642313.651676 s	max end: 1642313.655760 s				
perf:(2)	176.979 ms	317	avg: 0.098 ms	max: 3.121 ms	max
start: 1642315.522633 s	max end: 1642315.525754 s				
gmain:(3)	0.277 ms	13	avg: 0.085 ms	max: 1.004 ms	max
start: 1642312.136751 s	max end: 1642312.137755 s				
sshd-session:75619	5.827 ms	171	avg: 0.064 ms	max: 1.932 ms	max
start: 1642309.154822 s	max end: 1642309.156754 s				
ksoftirqd/3:37	0.015 ms	4	avg: 0.056 ms	max: 0.214 ms	max
start: 1642308.227771 s	max end: 1642308.227985 s				
bash:76710	2.525 ms	6	avg: 0.050 ms	max: 0.238 ms	max
start: 1642308.229870 s	max end: 1642308.230108 s				
kworker/2:2-mm_:67633	0.209 ms	34	avg: 0.045 ms	max: 1.001 ms	max
start: 1642310.968754 s	max end: 1642310.969755 s				
tail:76707	0.087 ms	3	avg: 0.043 ms	max: 0.114 ms	max
start: 1642311.527898 s	max end: 1642311.528012 s				
seq:76712	2.543 ms	4	avg: 0.036 ms	max: 0.113 ms	max
start: 1642308.227452 s	max end: 1642308.227565 s				
kworker/u18:3-e:74297	5.358 ms	69	avg: 0.032 ms	max: 0.292 ms	max
start: 1642315.940357 s	max end: 1642315.940649 s				
rcu_preempt:15	0.436 ms	112	avg: 0.030 ms	max: 1.004 ms	max
start: 1642309.721763 s	max end: 1642309.722767 s				
kworker/u19:1-f:66256	0.385 ms	27	avg: 0.018 ms	max: 0.260 ms	max
start: 1642313.272759 s	max end: 1642313.273019 s				
Bun Pool 2:75651	0.228 ms	18	avg: 0.016 ms	max: 0.054 ms	max
start: 1642309.713967 s	max end: 1642309.714020 s				
khugepaged:56	0.172 ms	1	avg: 0.016 ms	max: 0.016 ms	max
start: 1642314.936754 s	max end: 1642314.936769 s				
kworker/1:0H-kb:27	0.025 ms	1	avg: 0.013 ms	max: 0.013 ms	max
start: 1642313.913743 s	max end: 1642313.913757 s				
kworker/2:1H-kb:425	0.005 ms	1	avg: 0.013 ms	max: 0.013 ms	max
start: 1642315.449754 s	max end: 1642315.449768 s				
Bun Pool 0:75648	0.252 ms	20	avg: 0.012 ms	max: 0.029 ms	max
start: 1642309.713849 s	max end: 1642309.713878 s				
kworker/u20:2-e:67658	0.309 ms	24	avg: 0.010 ms	max: 0.014 ms	max
start: 1642309.291704 s	max end: 1642309.291717 s				
kworker/u18:1-e:74674	1.189 ms	14	avg: 0.009 ms	max: 0.013 ms	max
start: 1642308.792754 s	max end: 1642308.792766 s				
stress-ng-cpu:(3)	23386.393 ms	130849	avg: 0.009 ms	max: 4.004 ms	max
start: 1642309.735765 s	max end: 1642309.739769 s				
kworker/u17:4-e:74299	0.050 ms	5	avg: 0.009 ms	max: 0.011 ms	max
start: 1642308.513085 s	max end: 1642308.513096 s				
multipathd:(2)	0.374 ms	18	avg: 0.008 ms	max: 0.010 ms	max
start: 1642314.314092 s	max end: 1642314.314102 s				
chronyd:9319	0.525 ms	24	avg: 0.008 ms	max: 0.011 ms	max
start: 1642310.575489 s	max end: 1642310.575499 s				
kworker/u19:0-f:67224	0.201 ms	7	avg: 0.007 ms	max: 0.012 ms	max
start: 1642310.865619 s	max end: 1642310.865631 s				
kworker/0:0-mm_:74552	0.118 ms	14	avg: 0.005 ms	max: 0.007 ms	max

start: 1642311.535757 s		max end: 1642311.535763 s						
migration/0:18		0.034 ms		4		avg: 0.005 ms		max: 0.007 ms max
start: 1642316.237098 s		max end: 1642316.237105 s						
migration/3:36		0.019 ms		3		avg: 0.005 ms		max: 0.006 ms max
start: 1642316.242097 s		max end: 1642316.242103 s						
migration/2:30		0.031 ms		4		avg: 0.005 ms		max: 0.006 ms max
start: 1642312.241097 s		max end: 1642312.241103 s						
kworker/1:0-eve:67382		0.088 ms		14		avg: 0.005 ms		max: 0.008 ms max
start: 1642309.753753 s		max end: 1642309.753762 s						
kworker/3:0-mm_:67623		0.066 ms		10		avg: 0.005 ms		max: 0.006 ms max
start: 1642311.928754 s		max end: 1642311.928760 s						
migration/1:24		0.043 ms		5		avg: 0.004 ms		max: 0.006 ms max
start: 1642312.240097 s		max end: 1642312.240103 s						
kcompactd0:54		0.076 ms		16		avg: 0.004 ms		max: 0.005 ms max
start: 1642316.049754 s		max end: 1642316.049759 s						
lat_pipe:(9)		7072.855 ms		1544627		avg: 0.003 ms		max: 3.012 ms max
start: 1642311.725754 s		max end: 1642311.728766 s						
ksoftirqd/1:25		0.030 ms		6		avg: 0.003 ms		max: 0.003 ms max
start: 1642313.136764 s		max end: 1642313.136767 s						
ksoftirqd/2:31		0.029 ms		8		avg: 0.002 ms		max: 0.003 ms max
start: 1642313.950757 s		max end: 1642313.950760 s						
ksoftirqd/0:14		0.007 ms		1		avg: 0.002 ms		max: 0.002 ms max
start: 1642315.569754 s		max end: 1642315.569756 s						

TOTAL:		31227.662 ms		1684277	
--------	--	--------------	--	---------	--

INFO: 0.003% context switch bugs (24 out of 909885)

task1/p2b_sched_latency_noload.txt

Task	Runtime ms	Count	Avg delay ms	Max delay ms	Max
delay start	Max delay end				
kworker/1:0-eve:67382	0.050 ms	7	avg: 0.168 ms	max: 0.568 ms	max
start: 1642288.966757 s	max end: 1642288.967325 s				
kworker/2:2-eve:67633	0.130 ms	22	avg: 0.147 ms	max: 1.997 ms	max
start: 1642286.995758 s	max end: 1642286.997754 s				
kworker/u20:2-e:67658	0.117 ms	3	avg: 0.100 ms	max: 0.160 ms	max
start: 1642285.563755 s	max end: 1642285.563916 s				
Bun Pool 0:75648	0.169 ms	11	avg: 0.066 ms	max: 0.657 ms	max
start: 1642289.870115 s	max end: 1642289.870772 s				
JITWorker:76568	77.669 ms	47	avg: 0.050 ms	max: 1.046 ms	max
start: 1642287.871718 s	max end: 1642287.872764 s				
ksoftirqd/1:25	0.028 ms	2	avg: 0.049 ms	max: 0.094 ms	max
start: 1642285.441768 s	max end: 1642285.441862 s				
rcu_preempt:15	0.405 ms	92	avg: 0.047 ms	max: 1.003 ms	max
start: 1642286.993753 s	max end: 1642286.994756 s				
tail:76643	0.076 ms	3	avg: 0.033 ms	max: 0.083 ms	max
start: 1642289.625095 s	max end: 1642289.625178 s				
sshd-session:75619	3.295 ms	120	avg: 0.022 ms	max: 0.931 ms	max
start: 1642290.877843 s	max end: 1642290.878774 s				
Bun Pool 2:75651	0.153 ms	11	avg: 0.017 ms	max: 0.058 ms	max
start: 1642287.870183 s	max end: 1642287.870241 s				
kworker/3:1-mm_:75509	0.057 ms	8	avg: 0.017 ms	max: 0.088 ms	max
start: 1642288.952756 s	max end: 1642288.952844 s				
seq:76648	2.619 ms	2	avg: 0.016 ms	max: 0.022 ms	max
start: 1642285.441862 s	max end: 1642285.441884 s				
HeapHelper:(3)	65.326 ms	4157	avg: 0.014 ms	max: 2.311 ms	max
start: 1642289.651445 s	max end: 1642289.653756 s				
kworker/u18:1-e:74674	1.998 ms	45	avg: 0.012 ms	max: 0.073 ms	max
start: 1642288.319831 s	max end: 1642288.319904 s				
bash:76646	2.506 ms	6	avg: 0.011 ms	max: 0.015 ms	max
start: 1642289.625206 s	max end: 1642289.625221 s				
gmain:(3)	0.226 ms	5	avg: 0.011 ms	max: 0.012 ms	max
start: 1642286.137750 s	max end: 1642286.137763 s				
Bun Pool 3:75652	0.111 ms	9	avg: 0.010 ms	max: 0.027 ms	max
start: 1642286.474343 s	max end: 1642286.474369 s				
kworker/u18:3-e:74297	0.254 ms	10	avg: 0.010 ms	max: 0.016 ms	max
start: 1642286.777734 s	max end: 1642286.777750 s				
kcompactd0:54	0.064 ms	11	avg: 0.009 ms	max: 0.014 ms	max
start: 1642289.840753 s	max end: 1642289.840767 s				
multipathd:(2)	0.201 ms	10	avg: 0.009 ms	max: 0.012 ms	max
start: 1642288.313306 s	max end: 1642288.313318 s				
HTTP Client:75650	0.020 ms	1	avg: 0.008 ms	max: 0.008 ms	max
start: 1642288.371157 s	max end: 1642288.371165 s				
cron:1257	0.030 ms	1	avg: 0.008 ms	max: 0.008 ms	max
start: 1642290.034336 s	max end: 1642290.034345 s				
kworker/u19:0-e:67224	0.314 ms	15	avg: 0.008 ms	max: 0.012 ms	max
start: 1642285.582432 s	max end: 1642285.582444 s				
chronyd:9319	0.397 ms	17	avg: 0.008 ms	max: 0.010 ms	max
start: 1642285.937291 s	max end: 1642285.937301 s				
systemd-journal:66673	0.040 ms	1	avg: 0.007 ms	max: 0.007 ms	max
start: 1642285.603432 s	max end: 1642285.603439 s				
Bun Pool 1:75649	0.206 ms	15	avg: 0.007 ms	max: 0.013 ms	max
start: 1642285.474269 s	max end: 1642285.474282 s				
claude:(2)	311.599 ms	4454	avg: 0.007 ms	max: 0.710 ms	max
start: 1642285.449043 s	max end: 1642285.449754 s				
perf:(2)	99.929 ms	73	avg: 0.007 ms	max: 0.052 ms	max
start: 1642287.646754 s	max end: 1642287.646807 s				
kworker/u17:4-f:74299	0.013 ms	1	avg: 0.007 ms	max: 0.007 ms	max
start: 1642288.824754 s	max end: 1642288.824761 s				
kworker/0:0-mm_:74552	0.107 ms	14	avg: 0.007 ms	max: 0.024 ms	max

start: 1642287.932755 s		max end: 1642287.932779 s								
kworker/1:0H-kb:27		0.031 ms		1		avg: 0.006 ms		max: 0.006 ms		max
start: 1642288.314274 s		max end: 1642288.314279 s								
migration/0:18		0.019 ms		2		avg: 0.005 ms		max: 0.007 ms		max
start: 1642288.237098 s		max end: 1642288.237105 s								
migration/2:30		0.027 ms		3		avg: 0.005 ms		max: 0.006 ms		max
start: 1642288.241097 s		max end: 1642288.241103 s								
ksoftirqd/0:14		0.022 ms		4		avg: 0.005 ms		max: 0.009 ms		max
start: 1642288.952759 s		max end: 1642288.952768 s								
migration/3:36		0.019 ms		2		avg: 0.005 ms		max: 0.006 ms		max
start: 1642288.242097 s		max end: 1642288.242103 s								
lat_pipe:(9)		4675.642 ms		727548		avg: 0.004 ms		max: 1.449 ms		max
start: 1642286.189378 s		max end: 1642286.190827 s								
migration/1:24		0.014 ms		2		avg: 0.004 ms		max: 0.005 ms		max
start: 1642288.240097 s		max end: 1642288.240102 s								
kworker/2:1H-kb:425		0.005 ms		1		avg: 0.003 ms		max: 0.003 ms		max
start: 1642290.872753 s		max end: 1642290.872756 s								
ksoftirqd/3:37		0.010 ms		2		avg: 0.003 ms		max: 0.003 ms		max
start: 1642289.083761 s		max end: 1642289.083764 s								

TOTAL:		5243.903 ms		736738	
--------	--	-------------	--	--------	--

INFO: 0.000% context switch bugs (1 out of 689459)

task1/p2b_sched_timehist_load.txt

Samples of sched_switch event do not have callchains.

time	cpu	task name [tid/pid]	wait time (msec)	sch delay (msec)	run time (msec)
1642308.223803	[0000]	perf[76709]	0.000	0.000	0.000
1642308.223815	[0000]	migration/0[18]	0.000	0.003	0.012
1642308.224797	[0001]	perf[76709]	0.000	0.000	0.000
1642308.224808	[0001]	migration/1[24]	0.000	0.003	0.011
1642308.225795	[0002]	perf[76709]	0.000	0.000	0.000
1642308.225807	[0002]	migration/2[30]	0.000	0.003	0.011
1642308.225861	[0003]	perf[76709]	0.000	0.000	0.000
1642308.227494	[0003]	bash[76710]	0.000	0.005	1.633
1642308.227501	[0003]	rcu_preempt[15]	0.000	0.741	0.006
1642308.227565	[0003]	perf[76709]	1.639	1.639	0.064
1642308.227985	[0003]	seq[76712]	0.000	0.112	0.419
1642308.227988	[0003]	ksoftirqd/3[37]	0.000	0.213	0.003
1642308.228164	[0001]	stress-ng-cpu[76704]	0.000	0.000	3.356
1642308.228189	[0001]	sshd-session[75619]	0.000	0.006	0.024
1642308.229766	[0003]	seq[76712]	0.003	0.003	1.777
1642308.229772	[0003]	rcu_preempt[15]	2.264	0.012	0.006
1642308.230108	[0003]	seq[76712]	0.000	0.000	0.335
1642308.230131	[0003]	bash[76710]	2.613	0.238	0.023
1642308.230139	[0003]	seq[76712]	0.023	0.023	0.008
1642308.230303	[0003]	bash[76710]	0.008	0.005	0.164
1642308.231759	[0003]	lat_pipe[76713]	0.000	0.055	1.455
1642308.231765	[0003]	rcu_preempt[15]	1.986	0.005	0.005
1642308.234757	[0003]	lat_pipe[76713]	0.005	0.005	2.992
1642308.234760	[0003]	rcu_preempt[15]	2.992	0.004	0.002
1642308.236170	[0003]	lat_pipe[76713]	0.002	0.002	1.410
1642308.236190	[0003]	claude[75642]	0.000	0.005	0.019
1642308.236755	[0003]	lat_pipe[76713]	0.019	0.019	0.565
1642308.236758	[0003]	rcu_preempt[15]	1.995	0.002	0.002
1642308.236981	[0003]	lat_pipe[76713]	0.002	0.002	0.222
1642308.237040	[0003]	claude[75642]	0.790	0.004	0.059
1642308.237101	[0000]	stress-ng-cpu[76705]	0.000	0.000	13.286
1642308.237105	[0000]	migration/0[18]	13.286	0.004	0.003
1642308.238757	[0003]	lat_pipe[76713]	0.059	0.059	1.717
1642308.238760	[0003]	rcu_preempt[15]	1.999	0.004	0.002
1642308.240102	[0001]	stress-ng-cpu[76704]	0.024	0.024	11.912
1642308.240106	[0001]	migration/1[24]	15.294	0.004	0.003

task1/p2b_sched_timehist_noload.txt

Samples of sched_switch event do not have callchains.

time	cpu	task name [tid/pid]	wait time (msec)	sch delay (msec)	run time (msec)
1642285.439481	[0000]	perf[76645]	0.000	0.000	0.000
1642285.439492	[0000]	migration/0[18]	0.000	0.003	0.010
1642285.439529	[0001]	perf[76645]	0.000	0.000	0.000
1642285.439536	[0001]	migration/1[24]	0.000	0.002	0.007
1642285.439583	[0002]	perf[76645]	0.000	0.000	0.000
1642285.439594	[0002]	migration/2[30]	0.000	0.003	0.011
1642285.439683	[0000]	<idle>	0.000	0.000	0.190
1642285.439713	[0003]	perf[76645]	0.000	0.000	0.000
1642285.440754	[0002]	<idle>	0.000	0.000	1.159
1642285.440757	[0002]	rcu_preempt[15]	0.000	0.002	0.003
1642285.440769	[0002]	<idle>	0.003	0.003	0.012
1642285.440772	[0002]	rcu_preempt[15]	0.012	0.011	0.002
1642285.441342	[0001]	<idle>	0.000	0.000	1.806
1642285.441366	[0000]	bash[76646]	0.000	0.010	1.682
1642285.441862	[0001]	seq[76648]	0.000	0.010	0.519
1642285.441884	[0001]	ksoftirqd/1[25]	0.000	0.094	0.021
1642285.443727	[0000]	<idle>	1.682	1.682	2.361
1642285.443734	[0000]	bash[76646]	2.361	0.012	0.006
1642285.443959	[0000]	<idle>	0.006	0.006	0.224
1642285.443972	[0001]	seq[76648]	0.000	0.000	2.088
1642285.444083	[0002]	<idle>	0.002	0.002	3.310
1642285.444118	[0000]	bash[76646]	0.224	0.008	0.159
1642285.446484	[0003]	<idle>	0.000	0.000	6.771
1642285.446501	[0003]	chronyd[9319]	0.000	0.007	0.016
1642285.446756	[0002]	lat_pipe[76649]	0.000	0.010	2.673
1642285.446760	[0002]	rcu_preempt[15]	5.983	1.002	0.004
1642285.447757	[0002]	lat_pipe[76649]	0.004	0.004	0.997
1642285.447761	[0002]	rcu_preempt[15]	0.997	0.005	0.004
1642285.449754	[0002]	lat_pipe[76649]	0.004	0.004	1.992
1642285.449773	[0002]	claude[75642]	0.000	0.710	0.019
1642285.451757	[0002]	lat_pipe[76649]	0.019	0.019	1.983
1642285.451760	[0002]	rcu_preempt[15]	3.995	0.002	0.003
1642285.456754	[0002]	lat_pipe[76649]	0.003	0.003	4.994
1642285.456757	[0002]	rcu_preempt[15]	4.994	0.002	0.002
1642285.458762	[0002]	lat_pipe[76649]	0.002	0.002	2.004
1642285.458764	[0002]	rcu_preempt[15]	2.004	0.004	0.002

task1/p4_callgraph_noload.txt

```

# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 682K of event 'sched:sched_switch'
# Event count (approx.): 682111
#
# Children      Self  Trace output
# .....
#
16.63%    16.63% lat_pipe:77440 [120] S ==> swapper/0:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68f89
0x60deb1a64d26
0x60deb1a64be8
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

14.74%    14.74% lat_pipe:77439 [120] S ==> swapper/3:0 [120]
|
---0x60deb1a649e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main
0x60deb1a6496b
0x60deb1a693f4
0x60deb1a68fd4
0x60deb1a64b10
__libc_read (inlined)
__syscall_cancel
__internal_syscall_cancel (inlined)
entry_SYSCALL_64_after_hwframe
do_syscall_64
x64_sys_call
__x64_sys_read
ksys_read
vfs_read
anon_pipe_read
schedule
__schedule
perf_trace_sched_switch

12.93%    12.93% lat_pipe:77442 [120] S ==> swapper/0:0 [120]
|
---0x59949e3f59e5
__libc_start_main_impl (inlined)
call_init (inlined)
__libc_start_call_main

```



```
stdin:2:125-151: WARNING: Return value discarded.
tracepoint:sched:sched_switch /args->next_comm=="lat_pipe"/ { $s=@t[args->next_pid]; if($s){
@off_us=hist((nsecs-$s)/1000); delete(@t[args->next_pid]); } }
```

~~~~~

Attached 2 probes

Pipe latency: 17.5059 microseconds

@off\_us:

|              |        |                                                              |  |
|--------------|--------|--------------------------------------------------------------|--|
| [2, 4)       | 9993   | @@@                                                          |  |
| [4, 8)       | 353    |                                                              |  |
| [8, 16)      | 139094 | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ |  |
| [16, 32)     | 33289  | @@@@@@@@@@@@@@@@                                             |  |
| [32, 64)     | 944    |                                                              |  |
| [64, 128)    | 157    |                                                              |  |
| [128, 256)   | 36     |                                                              |  |
| [256, 512)   | 16     |                                                              |  |
| [512, 1K)    | 17     |                                                              |  |
| [1K, 2K)     | 5      |                                                              |  |
| [2K, 4K)     | 2      |                                                              |  |
| [4K, 8K)     | 0      |                                                              |  |
| [8K, 16K)    | 0      |                                                              |  |
| [16K, 32K)   | 0      |                                                              |  |
| [32K, 64K)   | 0      |                                                              |  |
| [64K, 128K)  | 0      |                                                              |  |
| [128K, 256K) | 0      |                                                              |  |
| [256K, 512K) | 0      |                                                              |  |
| [512K, 1M)   | 2      |                                                              |  |

@t[77549]: 1643768205293775

@t[77548]: 1643768206232093

@t[77535]: 1643768206467861

**task1/p4\_select\_idle\_sibling.txt**

```

static int select_idle_sibling(struct task_struct *p, int prev, int target)
{
    bool has_idle_core = false;
    struct sched_domain *sd;
    unsigned long task_util, util_min, util_max;
    int i, recent_used_cpu, prev_aff = -1;

    /*
     * On asymmetric system, update task utilization because we will check
     * that the task fits with CPU's capacity.
     */
    if (sched_asym_cpucap_active()) {
        sync_entity_load_avg(&p->se);
        task_util = task_util_est(p);
        util_min = uclamp_eff_value(p, UCLAMP_MIN);
        util_max = uclamp_eff_value(p, UCLAMP_MAX);
    }

    /*
     * per-cpu select_rq_mask usage
     */
    lockdep_assert_irqs_disabled();

    if ((available_idle_cpu(target) || sched_idle_cpu(target)) &&
        asym_fits_cpu(task_util, util_min, util_max, target))
        return target;

    /*
     * If the previous CPU is cache affine and idle, don't be stupid:
     */
    if (prev != target && cpus_share_cache(prev, target) &&
        (available_idle_cpu(prev) || sched_idle_cpu(prev)) &&
        asym_fits_cpu(task_util, util_min, util_max, prev)) {

        if (!static_branch_unlikely(&sched_cluster_active) ||
            cpus_share_resources(prev, target))
            return prev;

        prev_aff = prev;
    }

    /*
     * Allow a per-cpu kthread to stack with the wakee if the
     * kworker thread and the tasks previous CPUs are the same.
     * The assumption is that the wakee queued work for the
     * per-cpu kthread that is now complete and the wakeup is
     * essentially a sync wakeup. An obvious example of this
     * pattern is IO completions.
     */
    if (is_per_cpu_kthread(current) &&
        in_task() &&
        prev == smp_processor_id() &&
        this_rq()->nr_running <= 1 &&
        asym_fits_cpu(task_util, util_min, util_max, prev)) {
        return prev;
    }

    /* Check a recently used CPU as a potential idle candidate: */
    recent_used_cpu = p->recent_used_cpu;
    p->recent_used_cpu = prev;
    if (recent_used_cpu != prev &&
        recent_used_cpu != target &&
        cpus_share_cache(recent_used_cpu, target) &&
        (available_idle_cpu(recent_used_cpu) || sched_idle_cpu(recent_used_cpu)) &&
        cpumask_test_cpu(recent_used_cpu, p->cpus_ptr) &&
        asym_fits_cpu(task_util, util_min, util_max, recent_used_cpu)) {

```

```

        if (!static_branch_unlikely(&sched_cluster_active) ||
            cpus_share_resources(recent_used_cpu, target))
            return recent_used_cpu;

    } else {
        recent_used_cpu = -1;
    }

    /*
     * For asymmetric CPU capacity systems, our domain of interest is
     * sd_asym_cpucapacity rather than sd_llc.
     */
    if (sched_asym_cpucap_active()) {
        sd = rcu_dereference_all(per_cpu(sd_asym_cpucapacity, target));
        /*
         * On an asymmetric CPU capacity system where an exclusive
         * cpuset defines a symmetric island (i.e. one unique
         * capacity_orig value through the cpuset), the key will be set
         * but the CPUs within that cpuset will not have a domain with
         * SD_ASYM_CPUCAPACITY. These should follow the usual symmetric
         * capacity path.
         */
        if (sd) {
            i = select_idle_capacity(p, sd, target);
            return ((unsigned)i < nr_cpumask_bits) ? i : target;
        }
    }

    sd = rcu_dereference_all(per_cpu(sd_llc, target));
    if (!sd)
        return target;

    if (sched_smt_active()) {
        has_idle_core = test_idle_cores(target);

        if (!has_idle_core && cpus_share_cache(prev, target)) {
            i = select_idle_smt(p, sd, prev);
            if ((unsigned int)i < nr_cpumask_bits)
                return i;
        }
    }

    i = select_idle_cpu(p, sd, has_idle_core, target);
    if ((unsigned)i < nr_cpumask_bits)
        return i;

    /*
     * For cluster machines which have lower sharing cache like L2 or
     * LLC Tag, we tend to find an idle CPU in the target's cluster
     * first. But prev_cpu or recent_used_cpu may also be a good candidate,
     * use them if possible when no idle CPU found in select_idle_cpu().
     */
    if ((unsigned int)prev_aff < nr_cpumask_bits)
        return prev_aff;
    if ((unsigned int)recent_used_cpu < nr_cpumask_bits)
        return recent_used_cpu;

    return target;
}

```

**task1/p4\_select\_task\_rq\_fair.txt**

```

static int
select_task_rq_fair(struct task_struct *p, int prev_cpu, int wake_flags)
{
    int sync = (wake_flags & WF_SYNC) && !(current->flags & PF_EXITING);
    struct sched_domain *tmp, *sd = NULL;
    int cpu = smp_processor_id();
    int new_cpu = prev_cpu;
    int want_affine = 0;
    /* SD_flags and WF_flags share the first nibble */
    int sd_flag = wake_flags & 0xF;

    /*
     * required for stable ->cpus_allowed
     */
    lockdep_assert_held(&p->pi_lock);
    if (wake_flags & WF_TTWU) {
        record_wakee(p);

        if ((wake_flags & WF_CURRENT_CPU) &&
            cpumask_test_cpu(cpu, p->cpus_ptr))
            return cpu;

        if (!is_rd_overutilized(this_rq()->rd)) {
            new_cpu = find_energy_efficient_cpu(p, prev_cpu);
            if (new_cpu >= 0)
                return new_cpu;
            new_cpu = prev_cpu;
        }

        want_affine = !wake_wide(p) && cpumask_test_cpu(cpu, p->cpus_ptr);
    }

    rcu_read_lock();
    for_each_domain(cpu, tmp) {
        /*
         * If both 'cpu' and 'prev_cpu' are part of this domain,
         * cpu is a valid SD_WAKE_AFFINE target.
         */
        if (want_affine && (tmp->flags & SD_WAKE_AFFINE) &&
            cpumask_test_cpu(prev_cpu, sched_domain_span(tmp))) {
            if (cpu != prev_cpu)
                new_cpu = wake_affine(tmp, p, cpu, prev_cpu, sync);

            sd = NULL; /* Prefer wake_affine over balance flags */
            break;
        }

        /*
         * Usually only true for WF_EXEC and WF_FORK, as sched_domains
         * usually do not have SD_BALANCE_WAKE set. That means wakeup
         * will usually go to the fast path.
         */
        if (tmp->flags & sd_flag)
            sd = tmp;
        else if (!want_affine)
            break;
    }

    if (unlikely(sd)) {
        /* Slow path */
        new_cpu = sched_balance_find_dst_cpu(sd, p, cpu, prev_cpu, sd_flag);
    } else if (wake_flags & WF_TTWU) { /* XXX always ? */
        /* Fast path */
        new_cpu = select_idle_sibling(p, prev_cpu, new_cpu);
    }

    rcu_read_unlock();
}

```

```
    return new_cpu;  
}
```

## Task 2 — raw output

---

logs/task2\_20260610\_1658.log

Script started on 2026-06-10 16:58:57+00:00 [COMMAND="sh /tmp/task2\_phases.sh" <not executed on terminal>]

##### PHASE 0 – Environment #####

Linux course-08 7.0.0-15-generic #15-Ubuntu SMP PREEMPT\_DYNAMIC Wed Apr 22 16:06:43 UTC 2026  
x86\_64 GNU/Linux

|       | total | used  | free  | shared | buff/cache | available |
|-------|-------|-------|-------|--------|------------|-----------|
| Mem:  | 7.7Gi | 802Mi | 4.1Gi | 1.1Mi  | 3.1Gi      | 7.0Gi     |
| Swap: | 0B    | 0B    | 0B    |        |            |           |

kvm

tmpfs on /run type tmpfs (rw,nosuid,nodev,size=1625108k,nr\_inodes=819200,mode=755,inode64)  
/dev/vda1 on / type ext4 (rw,relatime,discard,errors=remount-ro,commit=30)  
devtmpfs on /dev type devtmpfs (rw,nosuid,size=3485632k,nr\_inodes=871408,mode=755,inode64)  
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,usrquota)  
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)  
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)  
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)  
cgroup2 on /sys/fs/cgroup type cgroup2  
(rw,nosuid,nodev,noexec,relatime,nsdelegate,memory\_recursiveprot,memory\_hugetlb\_accounting)  
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)  
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)  
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)  
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)  
systemd-1 on /proc/sys/fs/binfmt\_misc type autofs  
(rw,relatime,fd=36,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe\_ino=7472)  
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)  
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)  
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)  
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)  
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,size=4062772k,nr\_inodes=1048576,inode64,usrquota)  
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)  
/dev/vda13 on /boot type ext4 (rw,relatime)  
/dev/vda15 on /boot/efi type vfat  
(rw,relatime,fmask=0077,dmask=0077,codepage=437,iocharset=iso8859-1,shortname=mixed,errors=remount-ro)  
binfmt\_misc on /proc/sys/fs/binfmt\_misc type binfmt\_misc (rw,nosuid,nodev,noexec,relatime)  
none on /run/credentials/getty@tty1.service type tmpfs  
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr\_inodes=1024,mode=700,inode64,noswap)  
none on /run/credentials/serial-getty@ttyS0.service type tmpfs  
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr\_inodes=1024,mode=700,inode64,noswap)  
none on /run/credentials/systemd-journald.service type tmpfs  
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr\_inodes=1024,mode=700,inode64,noswap)  
none on /run/credentials/systemd-resolved.service type tmpfs  
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr\_inodes=1024,mode=700,inode64,noswap)  
none on /run/credentials/systemd-networkd.service type tmpfs  
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr\_inodes=1024,mode=700,inode64,noswap)  
tmpfs on /run/user/1000 type tmpfs  
(rw,nosuid,nodev,relatime,size=812552k,nr\_inodes=203138,mode=700,uid=1000,gid=1000,inode64)  
tracefs on /sys/kernel/debug/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)

THP:

[always] madvise never

sysbench:

sysbench 1.0.20

##### PHASE 1 – Setup + baseline #####

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

1 files, 65536Kb each, 64Mb total

Creating files for the test...

Extra file open flags: (none)

Creating file test\_file.0

67108864 bytes written in 0.05 seconds (1391.59 MiB/sec).

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

```
1 files, 65536Kb each, 64Mb total
Creating files for the test...
Extra file open flags: (none)
Creating file test_file.0
67108864 bytes written in 0.04 seconds (1791.93 MiB/sec).
--- iter 1 v1 ---
    reads/s:          1903871.72
    read, MiB/s:      7437.00
--- iter 1 v2 ---
    reads/s:          1928142.69
    read, MiB/s:      7531.81
--- iter 2 v1 ---
    reads/s:          1927429.41
    read, MiB/s:      7529.02
--- iter 2 v2 ---
    reads/s:          1931441.02
    read, MiB/s:      7544.69
--- iter 3 v1 ---
    reads/s:          1935611.44
    read, MiB/s:      7560.98
--- iter 3 v2 ---
    reads/s:          1877979.23
    read, MiB/s:      7335.86
--- iter 4 v1 ---
    reads/s:          1933791.32
    read, MiB/s:      7553.87
--- iter 4 v2 ---
    reads/s:          1935906.20
    read, MiB/s:      7562.13
--- iter 5 v1 ---
    reads/s:          1933905.20
    read, MiB/s:      7554.32
--- iter 5 v2 ---
    reads/s:          1931580.13
    read, MiB/s:      7545.23

Script done on 2026-06-10 16:59:47+00:00 [COMMAND_EXIT_CODE="0"]
```

logs/task2\_20260610\_1720\_p2.log

Script started on 2026-06-10 17:20:06+00:00 [COMMAND="sh /tmp/task2\_p2.sh" <not executed on terminal>]

##### Phase 1B sanity - filefrag #####

/home/ubuntu/v1/test\_file.0: 2 extents found

/home/ubuntu/v2/test\_file.0: 1 extent found

##### Phase 2a - perf stat, page-faults first #####

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1869152.62

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7301.38

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9349397

Latency (ms):

min: 0.00

avg: 0.00

max: 0.22

95th percentile: 0.00

sum: 3233.77

Threads fairness:

events (avg/stddev): 9349397.0000/0.00

execution time (avg/stddev): 3.2338/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

1813 page-faults

1813 minor-faults

8700007 dTLB-load-misses

13265901628 cycles

7472617351 instructions

5.008398862 seconds time elapsed

4.996943000 seconds user



0.008003000 seconds sys

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1894086.01

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7398.77

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9474102

Latency (ms):

min: 0.00

avg: 0.00

max: 2.31

95th percentile: 0.00

sum: 3211.32

Threads fairness:

events (avg/stddev): 9474102.0000/0.00

execution time (avg/stddev): 3.2113/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

|             |                  |
|-------------|------------------|
| 819         | page-faults      |
| 819         | minor-faults     |
| 8814990     | dTLB-load-misses |
| 13273661653 | cycles           |
| 7566095564  | instructions     |

5.008091487 seconds time elapsed

4.998933000 seconds user

0.002998000 seconds sys

##### Phase 2b – smaps huge mappings during run #####  
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:  
Number of threads: 1  
Initializing random number generator from current time

Extra file open flags: (none)  
1 files, 64MiB each  
64MiB total file size  
Block size 4KiB  
Number of IO requests: 0  
Read/Write ratio for combined random IO test: 1.50  
Periodic FSYNC enabled, calling fsync() each 100 requests.  
Calling fsync() at the end of test, Enabled.  
Using fast mmaped I/O mode  
Doing random read test  
Initializing worker threads...

Threads started!

|                  |          |
|------------------|----------|
| Rss:             | 74836 kB |
| Pss_File:        | 67939 kB |
| AnonHugePages:   | 0 kB     |
| ShmemPmdMapped:  | 0 kB     |
| FilePmdMapped:   | 0 kB     |
| Shared_Hugetlb:  | 0 kB     |
| Private_Hugetlb: | 0 kB     |

|                  |            |
|------------------|------------|
| File operations: |            |
| reads/s:         | 1901172.73 |
| writes/s:        | 0.00       |
| fsyncs/s:        | 0.00       |

|                 |         |
|-----------------|---------|
| Throughput:     |         |
| read, MiB/s:    | 7426.46 |
| written, MiB/s: | 0.00    |

|                         |         |
|-------------------------|---------|
| General statistics:     |         |
| total time:             | 5.0001s |
| total number of events: | 9509332 |

|                  |         |
|------------------|---------|
| Latency (ms):    |         |
| min:             | 0.00    |
| avg:             | 0.00    |
| max:             | 0.12    |
| 95th percentile: | 0.00    |
| sum:             | 3250.47 |

|                              |                   |
|------------------------------|-------------------|
| Threads fairness:            |                   |
| events (avg/stddev):         | 9509332.0000/0.00 |
| execution time (avg/stddev): | 3.2505/0.00       |

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:  
Number of threads: 1  
Initializing random number generator from current time

Extra file open flags: (none)  
1 files, 64MiB each  
64MiB total file size

Block size 4KiB  
Number of IO requests: 0  
Read/Write ratio for combined random IO test: 1.50  
Periodic FSYNC enabled, calling fsync() each 100 requests.  
Calling fsync() at the end of test, Enabled.  
Using fast mmaped I/O mode  
Doing random read test  
Initializing worker threads...

Threads started!

Rss: 74896 kB  
Pss\_File: 67960 kB  
AnonHugePages: 0 kB  
ShmemPmdMapped: 0 kB  
FilePmdMapped: 0 kB  
Shared\_Hugetlb: 0 kB  
Private\_Hugetlb: 0 kB

File operations:  
reads/s: 1928871.88  
writes/s: 0.00  
fsyncs/s: 0.00

Throughput:  
read, MiB/s: 7534.66  
written, MiB/s: 0.00

General statistics:  
total time: 5.0001s  
total number of events: 9647862

Latency (ms):  
min: 0.00  
avg: 0.00  
max: 0.11  
95th percentile: 0.00  
sum: 3228.15

Threads fairness:  
events (avg/stddev): 9647862.0000/0.00  
execution time (avg/stddev): 3.2281/0.00

##### Phase 2b – PMD vs base-page fault counts #####  
sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:  
Number of threads: 1  
Initializing random number generator from current time

Extra file open flags: (none)  
1 files, 64MiB each  
64MiB total file size  
Block size 4KiB  
Number of IO requests: 0  
Read/Write ratio for combined random IO test: 1.50  
Periodic FSYNC enabled, calling fsync() each 100 requests.  
Calling fsync() at the end of test, Enabled.  
Using fast mmaped I/O mode  
Doing random read test  
Initializing worker threads...

Threads started!

Attached 2 probes

@base: 1722

File operations:

|           |            |
|-----------|------------|
| reads/s:  | 1885992.60 |
| writes/s: | 0.00       |
| fsyncs/s: | 0.00       |

Throughput:

|                 |         |
|-----------------|---------|
| read, MiB/s:    | 7367.16 |
| written, MiB/s: | 0.00    |

General statistics:

|                         |         |
|-------------------------|---------|
| total time:             | 5.0001s |
| total number of events: | 9433394 |

Latency (ms):

|                  |         |
|------------------|---------|
| min:             | 0.00    |
| avg:             | 0.00    |
| max:             | 1.24    |
| 95th percentile: | 0.00    |
| sum:             | 3259.71 |

Threads fairness:

|                              |                   |
|------------------------------|-------------------|
| events (avg/stddev):         | 9433394.0000/0.00 |
| execution time (avg/stddev): | 3.2597/0.00       |

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

Attached 2 probes

@base: 1650

File operations:

|           |            |
|-----------|------------|
| reads/s:  | 1895122.69 |
| writes/s: | 0.00       |
| fsyncs/s: | 0.00       |

Throughput:

|                 |         |
|-----------------|---------|
| read, MiB/s:    | 7402.82 |
| written, MiB/s: | 0.00    |

General statistics:

```
total time:          5.0001s
total number of events: 9479040

Latency (ms):
  min:              0.00
  avg:              0.00
  max:              0.40
  95th percentile: 0.00
  sum:              3259.30

Threads fairness:
  events (avg/stddev): 9479040.0000/0.00
  execution time (avg/stddev): 3.2593/0.00

Script done on 2026-06-10 17:20:36+00:00 [COMMAND_EXIT_CODE="0"]
```

**task2/p0\_env.txt**

Linux course-08 7.0.0-15-generic #15-Ubuntu SMP PREEMPT\_DYNAMIC Wed Apr 22 16:06:43 UTC 2026  
x86\_64 GNU/Linux

|       | total | used  | free  | shared | buff/cache | available |
|-------|-------|-------|-------|--------|------------|-----------|
| Mem:  | 7.7Gi | 802Mi | 4.1Gi | 1.1Mi  | 3.1Gi      | 7.0Gi     |
| Swap: | 0B    | 0B    | 0B    |        |            |           |

kvm

```
tmpfs on /run type tmpfs (rw,nosuid,nodev,size=1625108k,nr_inodes=819200,mode=755,inode64)
/dev/vda1 on / type ext4 (rw,relatime,discard,errors=remount-ro,commit=30)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=3485632k,nr_inodes=871408,mode=755,inode64)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2
(rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs
(rw,relatime,fd=36,pgrp=1,timeout=0,minproto=5,maxproto=5,direct,pipe_ino=7472)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize=2M)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /tmp type tmpfs (rw,nosuid,nodev,size=4062772k,nr_inodes=1048576,inode64,usrquota)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)
/dev/vda13 on /boot type ext4 (rw,relatime)
/dev/vda15 on /boot/efi type vfat
(rw,relatime,fmask=0077,dmask=0077,codepage=437,iocharset=iso8859-
1,shortname=mixed,errors=remount-ro)
binfmt_misc on /proc/sys/fs/binfmt_misc type binfmt_misc (rw,nosuid,nodev,noexec,relatime)
none on /run/credentials/getty@tty1.service type tmpfs
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
none on /run/credentials/serial-getty@ttyS0.service type tmpfs
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
none on /run/credentials/systemd-journald.service type tmpfs
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
none on /run/credentials/systemd-resolved.service type tmpfs
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
none on /run/credentials/systemd-networkd.service type tmpfs
(ro,nosuid,nodev,noexec,relatime,nosymfollow,size=1024k,nr_inodes=1024,mode=700,inode64,noswap)
tmpfs on /run/user/1000 type tmpfs
(rw,nosuid,nodev,relatime,size=812552k,nr_inodes=203138,mode=700,uid=1000,gid=1000,inode64)
tracefs on /sys/kernel/debug/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
```

THP:

[always] madvise never

sysbench:

sysbench 1.0.20

task2/p10\_reboot\_confirmed.txt

```

# Task 2 – GAP REPRODUCED after fresh reboot (manual SSH session)
# 22-day uptime had fragmented memory → huge pages couldn't allocate → no PMD map → no gap.
# Fresh boot → memory defragmented → v2 folios PMD-mapped → gap appears. kernel 7.0.0-15-
generic.

=== fresh-boot baseline (verbatim assignment sequence) ===
v1 reads/s: 1,942,670
v2 reads/s: 2,124,442      (+9.4%)

=== perf stat v1 vs v2 (same RUN) ===
--- v1 ---
  reads/s:      1,886,428
  cycles:      13,255,912,155
  instructions:  7,538,218,104
  dTLB-load-misses: 8,790,240
  cache-misses:  130,604,123
  cache-references: 634,898,144
--- v2 ---
  reads/s:      2,082,856
  cycles:      13,263,522,195
  instructions:  8,306,823,225
  dTLB-load-misses: 18,488      <-- ~475x FEWER than v1
  cache-misses:  126,771,643    <-- ~equal to v1 (NOT the driver)
  cache-references: 676,552,167

=== smaps_rollup during a run (huge file mapping?) ===
--- v2 --- reads/s 2,121,280
  Rss:      74,872 kB
  FilePmdMapped: 65,536 kB      <-- whole 64 MiB file PMD (huge-page) mapped
--- v1 --- reads/s 1,938,905
  Rss:      74,972 kB
  FilePmdMapped: 0 kB          <-- v1 gets NO huge mapping

# CONFIRMED CHAIN: prepare 4M writes → 2MiB PMD-order folios → PMD huge-page mapping
# (FilePmdMapped=64MiB) → dTLB-misses collapse 8.79M→18K → +9-10% reads/s.
# It IS dTLB (cache-misses equal). CONFIG_READ_ONLY_THP_FOR_FS was a red herring; the path
# works here on fresh memory. Earlier non-reproduction = memory fragmentation (22d uptime).

```

**task2/p1\_baseline\_gap.txt**

```
--- iter 1 v1 ---
  reads/s:          1903871.72
  read, MiB/s:      7437.00
--- iter 1 v2 ---
  reads/s:          1928142.69
  read, MiB/s:      7531.81
--- iter 2 v1 ---
  reads/s:          1927429.41
  read, MiB/s:      7529.02
--- iter 2 v2 ---
  reads/s:          1931441.02
  read, MiB/s:      7544.69
--- iter 3 v1 ---
  reads/s:          1935611.44
  read, MiB/s:      7560.98
--- iter 3 v2 ---
  reads/s:          1877979.23
  read, MiB/s:      7335.86
--- iter 4 v1 ---
  reads/s:          1933791.32
  read, MiB/s:      7553.87
--- iter 4 v2 ---
  reads/s:          1935906.20
  read, MiB/s:      7562.13
--- iter 5 v1 ---
  reads/s:          1933905.20
  read, MiB/s:      7554.32
--- iter 5 v2 ---
  reads/s:          1931580.13
  read, MiB/s:      7545.23
```

#### **task2/plb\_filefrag.txt**

```
/home/ubuntu/v1/test_file.0: 2 extents found
/home/ubuntu/v2/test_file.0: 1 extent found
```

#### **task2/p2\_faultcount\_v1.txt**

Attached 2 probes

@base: 1700

#### **task2/p2\_faultcount\_v2.txt**

Attached 2 probes

@base: 1694

#### **task2/p2\_perfstat\_v1.txt**



sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1882830.28

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7354.81

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9417768

Latency (ms):

min: 0.00

avg: 0.00

max: 0.93

95th percentile: 0.00

sum: 3221.33

Threads fairness:

events (avg/stddev): 9417768.0000/0.00

execution time (avg/stddev): 3.2213/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

|             |                  |
|-------------|------------------|
| 1813        | page-faults      |
| 1813        | minor-faults     |
| 8744015     | dTLB-load-misses |
| 13255640893 | cycles           |
| 7529345671  | instructions     |

5.008987211 seconds time elapsed

4.992305000 seconds user

0.008995000 seconds sys

task2/p2\_perfststat\_v2.txt

sysbench 1.0.20 (using system LuaJIT 2.1.1761786044)

Running the test with following options:

Number of threads: 1

Initializing random number generator from current time

Extra file open flags: (none)

1 files, 64MiB each

64MiB total file size

Block size 4KiB

Number of IO requests: 0

Read/Write ratio for combined random IO test: 1.50

Periodic FSYNC enabled, calling fsync() each 100 requests.

Calling fsync() at the end of test, Enabled.

Using fast mmaped I/O mode

Doing random read test

Initializing worker threads...

Threads started!

File operations:

reads/s: 1897579.84

writes/s: 0.00

fsyncs/s: 0.00

Throughput:

read, MiB/s: 7412.42

written, MiB/s: 0.00

General statistics:

total time: 5.0002s

total number of events: 9491604

Latency (ms):

min: 0.00

avg: 0.00

max: 0.22

95th percentile: 0.00

sum: 3214.20

Threads fairness:

events (avg/stddev): 9491604.0000/0.00

execution time (avg/stddev): 3.2142/0.00

Performance counter stats for 'sysbench fileio --file-num=1 --file-total-size=64M --file-test-mode=rndrd --file-io-mode=mmap --file-block-size=4K --time=5 run':

|             |                  |
|-------------|------------------|
| 821         | page-faults      |
| 821         | minor-faults     |
| 8754813     | dTLB-load-misses |
| 13272642788 | cycles           |
| 7576257521  | instructions     |

5.007510411 seconds time elapsed

4.998731000 seconds user

0.006997000 seconds sys

task2/p2\_smmaps\_v1.txt

```
Rss:                74900 kB
Pss_File:           67913 kB
AnonHugePages:      0 kB
ShmemPmdMapped:     0 kB
FilePmdMapped:      0 kB
Shared_Hugetlb:     0 kB
Private_Hugetlb:    0 kB
```

#### **task2/p2\_smaps\_v2.txt**

```
Rss:                74904 kB
Pss_File:           67909 kB
AnonHugePages:      0 kB
ShmemPmdMapped:     0 kB
FilePmdMapped:      0 kB
Shared_Hugetlb:     0 kB
Private_Hugetlb:    0 kB
```

#### **task2/p3\_bigfile.txt**

```
=== 1G v1 ===
reads/s:                1433224.31
read, MiB/s:            5598.53
      17173      page-faults
      7408592    dTLB-load-misses
=== 1G v2 ===
reads/s:                1455012.84
read, MiB/s:            5683.64
      1301      page-faults
      7413252    dTLB-load-misses
```

#### **task2/p3\_folios\_v1.txt**

Attached 1 probe

@folios: 7265

#### **task2/p3\_folios\_v2.txt**

Attached 1 probe

@folios: 3228

#### **task2/p3\_thp\_sweep.txt**

```
=== THP=always ===  
v1 1906808.01  
v2 1917326.41  
v1 1900403.36  
v2 1925281.76  
v1 1900552.16  
v2 1922220.71  
=== THP=advise ===  
v1 1903404.33  
v2 1926935.82  
v1 1904164.73  
v2 1926012.28  
v1 1907372.18  
v2 1926641.79  
=== THP=never ===  
v1 1906082.91  
v2 1930129.17  
v1 1908960.94  
v2 1927254.75  
v1 1906087.59  
v2 1947566.74
```

task2/p4\_do\_set\_pmd.txt

```

vm_fault_t do_set_pmd(struct vm_fault *vmf, struct folio *folio, struct page *page)
{
    struct vm_area_struct *vma = vmf->vma;
    bool write = vmf->flags & FAULT_FLAG_WRITE;
    unsigned long haddr = vmf->address & HPAGE_PMD_MASK;
    pmd_t entry;
    vm_fault_t ret = VM_FAULT_FALLBACK;

    /*
     * It is too late to allocate a small folio, we already have a large
     * folio in the pagecache: especially s390 KVM cannot tolerate any
     * PMD mappings, but PTE-mapped THP are fine. So let's simply refuse any
     * PMD mappings if THPs are disabled. As we already have a THP,
     * behave as if we are forcing a collapse.
     */
    if (thp_disabled_by_hw() || vma_thp_disabled(vma, vma->vm_flags,
        /* forced_collapse=*/ true))
        return ret;

    if (!thp_vma_suitable_order(vma, haddr, PMD_ORDER))
        return ret;

    if (folio_order(folio) != HPAGE_PMD_ORDER)
        return ret;
    page = &folio->page;

    /*
     * Just backoff if any subpage of a THP is corrupted otherwise
     * the corrupted page may mapped by PMD silently to escape the
     * check. This kind of THP just can be PTE mapped. Access to
     * the corrupted subpage should trigger SIGBUS as expected.
     */
    if (unlikely(folio_test_has_hwpoisoned(folio)))
        return ret;

    /*
     * Archs like ppc64 need additional space to store information
     * related to pte entry. Use the preallocated table for that.
     */
    if (arch_needs_pgtable_deposit() && !vmf->prealloc_pte) {
        vmf->prealloc_pte = pte_alloc_one(vma->vm_mm);
        if (!vmf->prealloc_pte)
            return VM_FAULT_OOM;
    }

    vmf->ptl = pmd_lock(vma->vm_mm, vmf->pmd);
    if (unlikely(!pmd_none(*vmf->pmd)))
        goto out;

    flush_icache_pages(vma, page, HPAGE_PMD_NR);

    entry = folio_mk_pmd(folio, vma->vm_page_prot);
    if (write)
        entry = maybe_pmd_mkdirty(pmd_mkdirty(entry), vma);

    add_mm_counter(vma->vm_mm, mm_counter_file(folio), HPAGE_PMD_NR);
    folio_add_file_rmap_pmd(folio, page, vma);

    /*
     * deposit and withdraw with pmd lock held
     */
    if (arch_needs_pgtable_deposit())
        deposit_prealloc_pte(vmf);

    set_pmd_at(vma->vm_mm, haddr, vmf->pmd, entry);
}

```

```
    update_mmu_cache_pmd(vma, haddr, vmf->pmd);

    /* fault is handled */
    ret = 0;
    count_vm_event(THP_FILE_MAPPED);
out:
    spin_unlock(vmf->ptl);
    return ret;
}
```

#### **task2/p4\_kconfig\_thp.txt**

```
CONFIG_HAVE_ARCH_TRANSPARENT_HUGEPAGE=y
CONFIG_HAVE_ARCH_TRANSPARENT_HUGEPAGE_PUD=y
CONFIG_TRANSPARENT_HUGEPAGE=y
# CONFIG_TRANSPARENT_HUGEPAGE_ALWAYS is not set
CONFIG_TRANSPARENT_HUGEPAGE_MADVISE=y
# CONFIG_TRANSPARENT_HUGEPAGE_NEVER is not set
CONFIG_TRANSPARENT_HUGEPAGE_SHMEM_HUGE_NEVER=y
# CONFIG_TRANSPARENT_HUGEPAGE_SHMEM_HUGE_ALWAYS is not set
# CONFIG_TRANSPARENT_HUGEPAGE_SHMEM_HUGE_WITHIN_SIZE is not set
# CONFIG_TRANSPARENT_HUGEPAGE_SHMEM_HUGE_ADVISE is not set
CONFIG_TRANSPARENT_HUGEPAGE_TMPFS_HUGE_NEVER=y
# CONFIG_TRANSPARENT_HUGEPAGE_TMPFS_HUGE_ALWAYS is not set
# CONFIG_TRANSPARENT_HUGEPAGE_TMPFS_HUGE_WITHIN_SIZE is not set
# CONFIG_TRANSPARENT_HUGEPAGE_TMPFS_HUGE_ADVISE is not set
# CONFIG_READ_ONLY_THP_FOR_FS is not set
```

#### **task2/p4\_page\_cache\_ra\_order.txt**

```

void page_cache_ra_order(struct readahead_control *ractl,
                        struct file_ra_state *ra)
{
    struct address_space *mapping = ractl->mapping;
    pgoff_t start = readahead_index(ractl);
    pgoff_t index = start;
    unsigned int min_order = mapping_min_folio_order(mapping);
    pgoff_t limit = (i_size_read(mapping->host) - 1) >> PAGE_SHIFT;
    pgoff_t mark = index + ra->size - ra->async_size;
    unsigned int nofs;
    int err = 0;
    gfp_t gfp = readahead_gfp_mask(mapping);
    unsigned int new_order = ra->order;

    trace_page_cache_ra_order(mapping->host, start, ra);
    if (!mapping_large_folio_support(mapping)) {
        ra->order = 0;
        goto fallback;
    }

    limit = min(limit, index + ra->size - 1);

    new_order = min(mapping_max_folio_order(mapping), new_order);
    new_order = min_t(unsigned int, new_order, ilog2(ra->size));
    new_order = max(new_order, min_order);

    ra->order = new_order;

    /* See comment in page_cache_ra_unbounded() */
    nofs = memalloc_nofs_save();
    filemap_invalidate_lock_shared(mapping);
    /*
     * If the new_order is greater than min_order and index is
     * already aligned to new_order, then this will be noop as index
     * aligned to new_order should also be aligned to min_order.
     */
    ractl->_index = mapping_align_index(mapping, index);
    index = readahead_index(ractl);

    while (index <= limit) {
        unsigned int order = new_order;

        /* Align with smaller pages if needed */
        if (index & ((1UL << order) - 1))
            order = __ffs(index);
        /* Don't allocate pages past EOF */
        while (order > min_order && index + (1UL << order) - 1 > limit)
            order--;
        err = ra_alloc_folio(ractl, index, mark, order, gfp);
        if (err)
            break;
        index += 1UL << order;
    }

    read_pages(ractl);
    filemap_invalidate_unlock_shared(mapping);
    memalloc_nofs_restore(nofs);

    /*
     * If there were already pages in the page cache, then we may have
     * left some gaps. Let the regular readahead code take care of this
     * situation below.
     */
    if (!err)
        return;
    fallback:

```







```
# 5b - anon random-read, THP ON vs OFF, dTLB-load-misses + throughput
# program controls THP per-mapping via madvise(MADV_HUGEPAGE/NOHUGEPAGE); 200M random 4K-strided reads.
# global THP: [always] madvise never    kernel: 7.0.0-15-generic    Sun Jun 14 11:42:54 UTC 2026
```

```
=== 64MB huge=0 ===
```

```
MB=64 huge=0 reads/s=89734904 sum=200000000
```

```
Performance counter stats for '/tmp/thp_rndrd 64 0':
```

|            |                  |
|------------|------------------|
| 178044444  | dTLB-load-misses |
| 6031323640 | cycles           |
| 3485579446 | instructions     |

```
2.275540705 seconds time elapsed
```

```
2.233620000 seconds user
```

```
0.040011000 seconds sys
```

```
=== 64MB huge=1 ===
```

```
MB=64 huge=1 reads/s=92780363 sum=200000000
```

```
Performance counter stats for '/tmp/thp_rndrd 64 1':
```

|            |                  |
|------------|------------------|
| 178067422  | dTLB-load-misses |
| 5835230177 | cycles           |
| 3485566318 | instructions     |

```
2.200230156 seconds time elapsed
```

```
2.153529000 seconds user
```

```
0.045990000 seconds sys
```

```
=== 1024MB huge=0 ===
```

```
MB=1024 huge=0 reads/s=57789639 sum=200000000
```

```
Performance counter stats for '/tmp/thp_rndrd 1024 0':
```

|             |                  |
|-------------|------------------|
| 198811347   | dTLB-load-misses |
| 10882728665 | cycles           |
| 4550870680  | instructions     |

```
4.145990515 seconds time elapsed
```

```
3.513013000 seconds user
```

```
0.631916000 seconds sys
```

```
=== 1024MB huge=1 ===
```

```
MB=1024 huge=1 reads/s=57987552 sum=200000000
```

```
Performance counter stats for '/tmp/thp_rndrd 1024 1':
```

|             |                  |
|-------------|------------------|
| 198705721   | dTLB-load-misses |
| 10848295121 | cycles           |
| 4544993855  | instructions     |

```
4.133136746 seconds time elapsed
```

```
3.490054000 seconds user
```

```
0.641852000 seconds sys
```

```
# =====
# SANITY CHECK – did THP actually map? (the runbook's read depends on this)
# =====
# The runbook's sanity line greps /proc/self/smaps_rollup – but that is the
# grep/shell process, NOT the test, so it can never confirm the test's mapping.
# Corrected check below inspects the test program's OWN mapping.
#
# Result: AnonHugePages = 0 kB for BOTH huge=1 and huge=0, at 64 MiB and 1 GiB.
# - polled test's /proc/PID/smaps_rollup mid-run (1024MB huge=1): AnonHugePages 0 kB
# - self-inspecting probe right after memset (64MB): huge=1 -> 0 kB, huge=0 -> 0 kB
#
# => THP was NEVER mapped. The MADV_HUGEPAGE "huge=1" run used 4 KiB base pages,
# same as huge=0. That is WHY dTLB-load-misses are identical (178M/178M at 64M,
# 198M/198M at 1G) and reads/s barely move (+3% / +0.3%).
#
# CONCLUSION: 5b as written is INCONCLUSIVE on this kernel – the dTLB/huge-page
# lever was never actually pulled, so this run neither proves NOR disproves the
# dTLB story. It is NOT evidence that the dTLB lever is worthless.
#
# Why THP did not map (environment facts gathered, all point AT the kernel, not the test):
# - mmap base IS 2 MiB-aligned (0x...600000) -> alignment not the cause
# - top-level enabled = [always] -> THP on as shipped
# - hugepages-2048kB/enabled = [inherit] -> follows always-> PMD mTHP enabled
# - defrag = [madvise] (as-shipped); also retried defrag=always + compact_memory
# - free memory 6.6 GiB -> not a fragmentation/OOM block
# - global anon THP IS functional: vmstat thp_fault_alloc=6634, thp_fault_fallback=0
# - BUT our mapping records ZERO thp_fault_alloc delta + ZERO fallback
# -> the anon fault path is not even ATTEMPTING a huge page for this mmap.
#
# OPEN QUESTION for the student (final connection, per CLAUDE.md):
# why does the fault path skip PMD-THP for a 2M-aligned MADV_HUGEPAGE anon region
# on kernel 7.0.0-15 when it succeeds for other processes? (khugepaged-over-time
# collapse not yet tested – interrupted.) Until 5b actually maps THP, the
# "huge-page dTLB win" claim from Phase 4 remains UNPROVEN here.
#
# (knobs left as-shipped: enabled=always, defrag=madvise)
```

**task2/p5c\_force\_thp.txt**

=== 256MB mode=0 ===

mode=0 MB=256 AnonHugePages\_kB=0

mode=0 MB=256 reads/s=62759989 sum=200000000

Performance counter stats for '/tmp/thp\_rndrd2 256 0':

|            |                  |
|------------|------------------|
| 194681566  | dTLB-load-misses |
| 8881399109 | cycles           |
| 3731522806 | instructions     |

3.359090688 seconds time elapsed

3.192268000 seconds user

0.164014000 seconds sys

=== 256MB mode=1 ===

MADV\_COLLAPSE failed: Invalid argument

mode=1 MB=256 AnonHugePages\_kB=0

mode=1 MB=256 reads/s=62624913 sum=200000000

Performance counter stats for '/tmp/thp\_rndrd2 256 1':

|            |                  |
|------------|------------------|
| 194680010  | dTLB-load-misses |
| 8903528253 | cycles           |
| 3725908083 | instructions     |

3.365086766 seconds time elapsed

3.205633000 seconds user

0.158970000 seconds sys

=== 1024MB mode=0 ===

mode=0 MB=1024 AnonHugePages\_kB=0

mode=0 MB=1024 reads/s=56452905 sum=200000000

Performance counter stats for '/tmp/thp\_rndrd2 1024 0':

|             |                  |
|-------------|------------------|
| 198810736   | dTLB-load-misses |
| 11112291706 | cycles           |
| 4656052798  | instructions     |

4.230569558 seconds time elapsed

3.592510000 seconds user

0.636054000 seconds sys

=== 1024MB mode=1 ===

MADV\_COLLAPSE failed: Invalid argument

mode=1 MB=1024 AnonHugePages\_kB=0

mode=1 MB=1024 reads/s=56409139 sum=200000000

Performance counter stats for '/tmp/thp\_rndrd2 1024 1':

|             |                  |
|-------------|------------------|
| 198815712   | dTLB-load-misses |
| 11153220204 | cycles           |
| 4654098855  | instructions     |

4.242477699 seconds time elapsed

3.609043000 seconds user

0.632823000 seconds sys

```
HugePages_Total:    600
HugePages_Free:    600
=== 256MB mode=2 (MAP_HUGETLB) ===
mode=2 MB=256 AnonHugePages_kB=0 (hugetlb: see HugePages_Free)
mode=2 MB=256 reads/s=70738072 sum=200000000
```

Performance counter stats for '/tmp/thp\_rndrd2 256 2':

|            |                  |
|------------|------------------|
| 2753       | dTLB-load-misses |
| 7657971203 | cycles           |
| 3429336113 | instructions     |

2.883778716 seconds time elapsed

2.859685000 seconds user  
0.020998000 seconds sys

```
=== 1024MB mode=2 (MAP_HUGETLB) ===
mode=2 MB=1024 AnonHugePages_kB=0 (hugetlb: see HugePages_Free)
mode=2 MB=1024 reads/s=67850882 sum=200000000
```

Performance counter stats for '/tmp/thp\_rndrd2 1024 2':

|            |                  |
|------------|------------------|
| 13578      | dTLB-load-misses |
| 8412641647 | cycles           |
| 3455270546 | instructions     |

3.159761606 seconds time elapsed

3.075510000 seconds user  
0.081935000 seconds sys